



National University
of computer and emerging sciences

MLOPS

Assignment #1

Usman Haroon

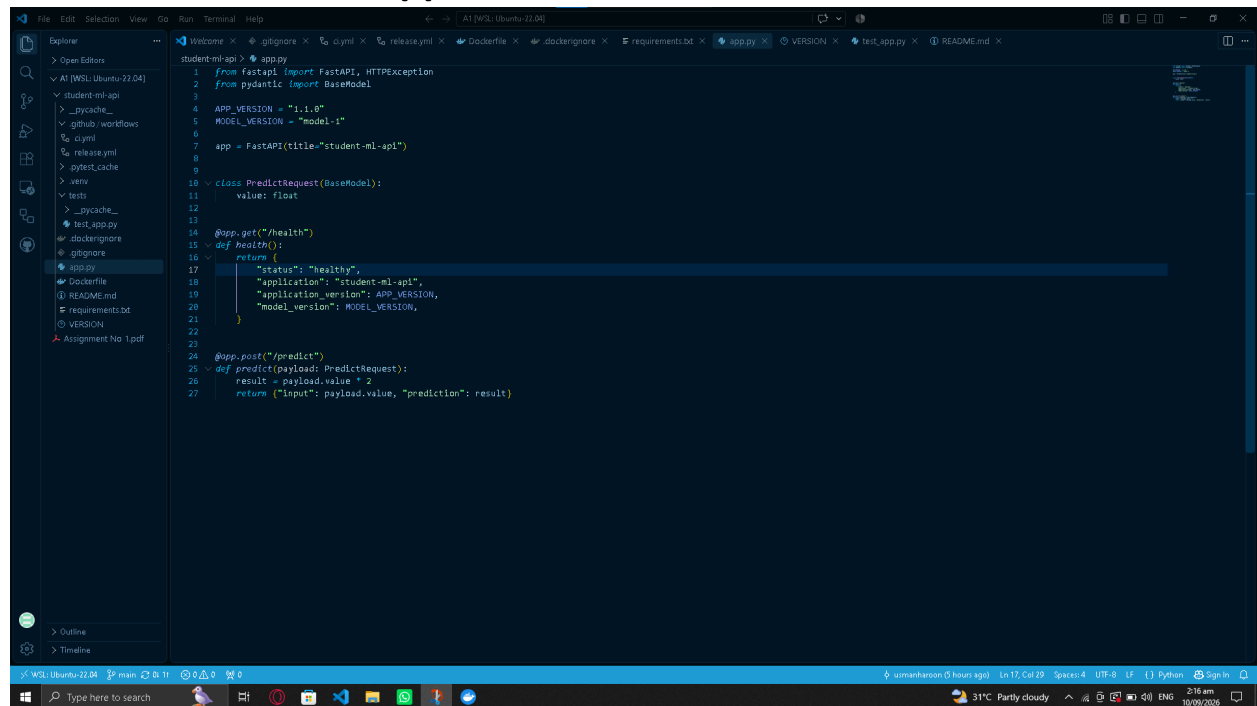
22i-1177

CS-B

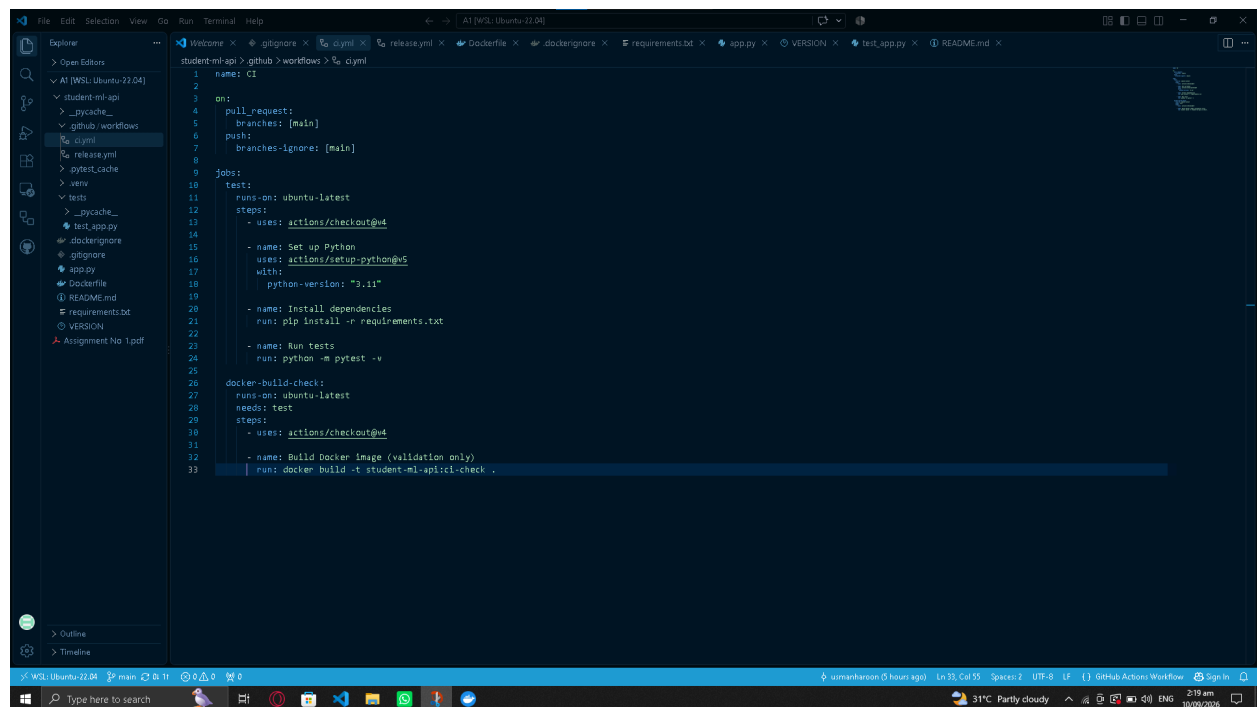
Submitted to:

Sir Hammad Majeed

Part 1 — Create the Application



```
1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel
3
4 APP_VERSION = "1.1.0"
5 MODEL_VERSION = "model-1"
6
7 app = FastAPI(title="student-ml-api")
8
9
10 class PredictRequest(BaseModel):
11     value: float
12
13
14 @app.get("/health")
15 def health():
16     return {
17         "status": "healthy",
18         "application": "student-ml-api",
19         "application_version": APP_VERSION,
20         "model_version": MODEL_VERSION,
21     }
22
23
24 @app.post("/predict")
25 def predict(payload: PredictRequest):
26     result = payload.value * 2
27     return {"input": payload.value, "prediction": result}
```

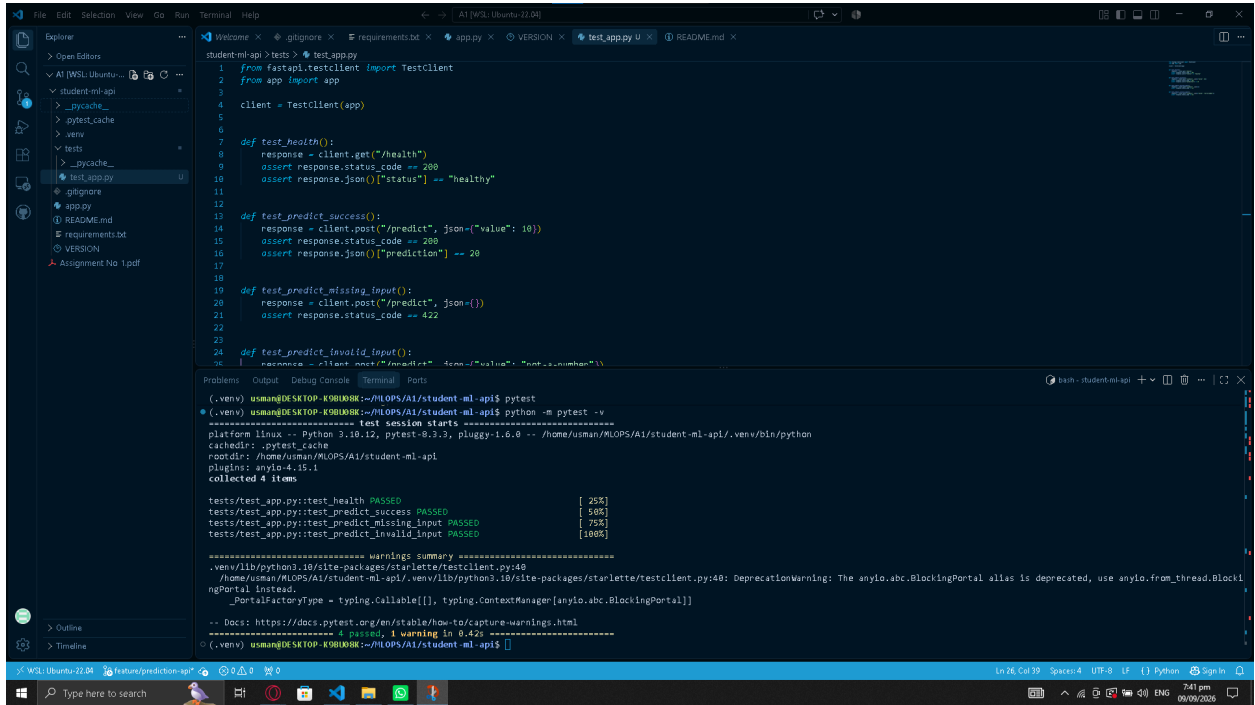


```
1 name: CI
2
3 on:
4   pull_request:
5     branches: [main]
6   push:
7     branches-ignore: [main]
8
9 jobs:
10   test:
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@v4
14
15       - name: Set up Python
16         uses: actions/setup-python@v5
17         with:
18           python-version: "3.11"
19
20       - name: Install dependencies
21         run: pip install -r requirements.txt
22
23       - name: Run tests
24         run: python -m pytest -v
25
26   docker-build-check:
27     runs-on: ubuntu-latest
28     needs: test
29     steps:
30       - uses: actions/checkout@v4
31
32       - name: Build Docker image (validation only)
33         run: docker build -t student-ml-api:ci-check .
```

Code of app.py and ci.yml

Part 2 — Add Automated Tests

Local pytest run



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with files like `student_ml_api`, `tests`, `requirements.txt`, `VERSION`, and `README.md`. The `tests` directory is expanded, showing `test_app.py`. The `test_app.py` file is open in the editor, showing the following code:

```
1 from fastapi.testclient import TestClient
2 from app import app
3
4 client = TestClient(app)
5
6
7 def test_health():
8     response = client.get("/health")
9     assert response.status_code == 200
10    assert response.json()["status"] == "healthy"
11
12
13 def test_predict_success():
14     response = client.post("/predict", json={"value": 10})
15     assert response.status_code == 200
16     assert response.json()["prediction"] == 20
17
18
19 def test_predict_missing_input():
20     response = client.post("/predict", json={})
21     assert response.status_code == 422
22
23
24 def test_predict_invalid_input():
25     response = client.post("/predict", json={"value": "not_a_number"})
```

The terminal output shows the command `pytest` being run, and the output indicates that all four tests passed:

```
(.venv) usman@DESKTOP-K9B8H8K:~/MLOPS/AI/student-ml-api$ pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest 8.3.3, pluggy 1.6.0 -- /home/usman/MLOPS/AI/student-ml-api/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/usman/MLOPS/AI/student-ml-api
plugins: anyio-4.5.1
collected 4 items

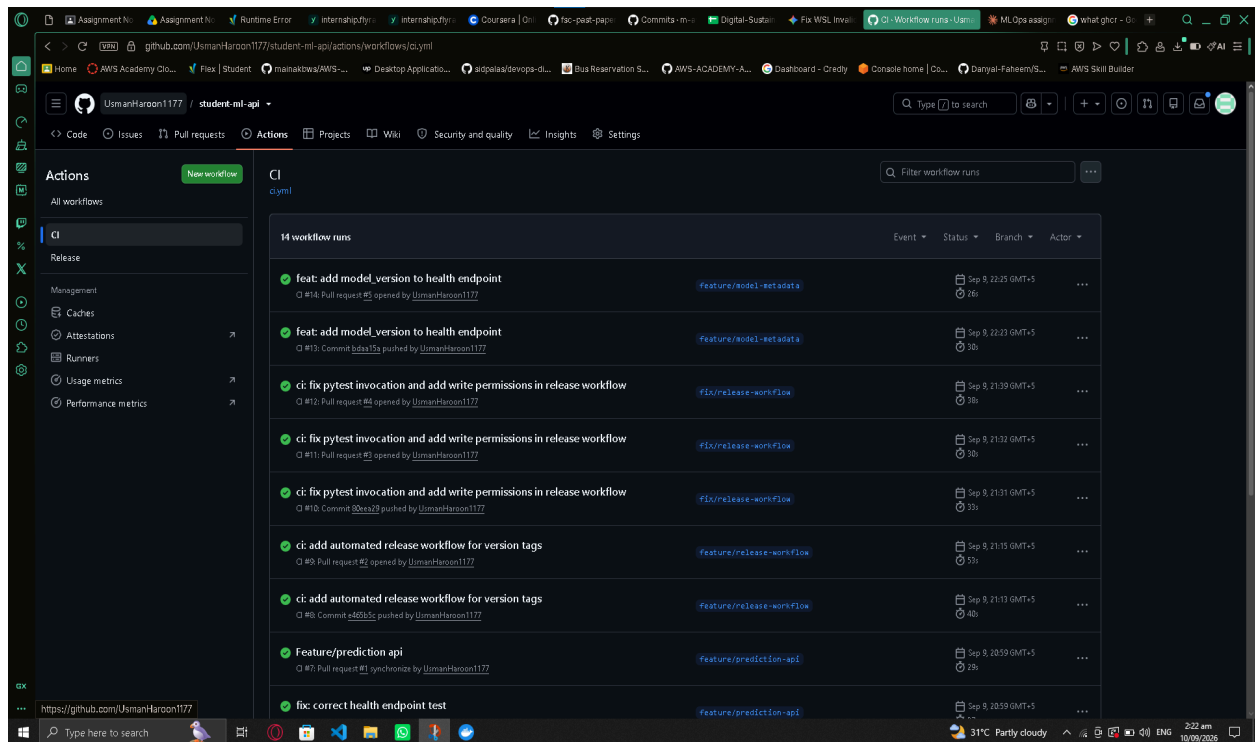
tests/test_app.py::test_health PASSED [ 25%]
tests/test_app.py::test_predict_success PASSED [ 50%]
tests/test_app.py::test_predict_missing_input PASSED [ 75%]
tests/test_app.py::test_predict_invalid_input PASSED [100%]

===== warnings summary =====
/home/usman/MLOPS/AI/student-ml-api/.venv/lib/python3.10/site-packages/starlette/testclient.py:48: DeprecationWarning: The anyio.bcp.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = typing.Callable[[], typing.ContextManager[anyio.bcp.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 1 warning in 0.42s =====
(.venv) usman@DESKTOP-K9B8H8K:~/MLOPS/AI/student-ml-api$
```

Terminal output right after running pytest before anything was pushed. Confirms all four tests pass locally before CI is even involved.

Part 3 — Establish a Professional Git Workflow



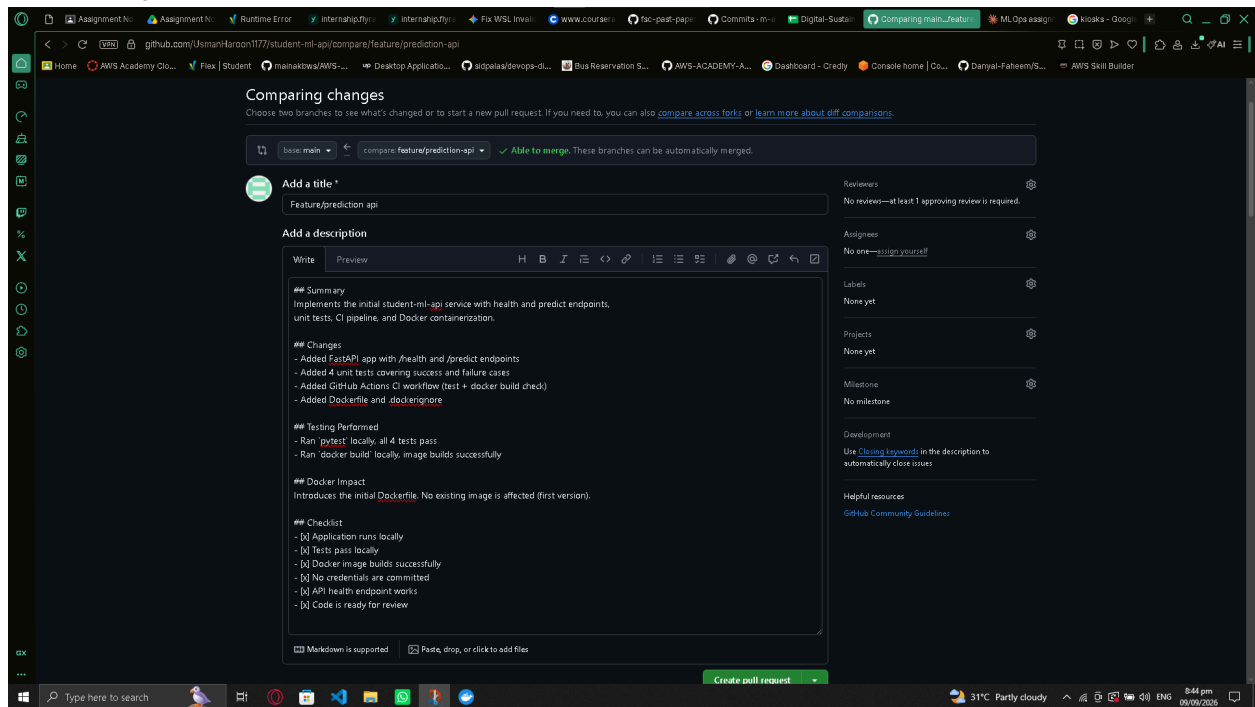
The screenshot displays the GitHub Actions interface for the repository 'student-mi-api'. The left sidebar shows the 'Actions' tab with a 'New workflow' button. The main area lists 14 workflow runs, each with a green checkmark indicating success. The runs are organized by event type: 'feat: add model_version to health endpoint', 'ci: fix pytest invocation and add write permissions in release workflow', 'ci: add automated release workflow for version tags', 'Feature/prediction api', and 'fix correct health endpoint test'. Each run includes details such as the event type, the workflow name, the branch, the actor, and the time taken to complete.

Event	Status	Branch	Actor
feat: add model_version to health endpoint	Success	feature/model-version	UmanHaroon1177
feat: add model_version to health endpoint	Success	feature/model-version	UmanHaroon1177
ci: fix pytest invocation and add write permissions in release workflow	Success	fix/release-workflow	UmanHaroon1177
ci: fix pytest invocation and add write permissions in release workflow	Success	fix/release-workflow	UmanHaroon1177
ci: fix pytest invocation and add write permissions in release workflow	Success	fix/release-workflow	UmanHaroon1177
ci: add automated release workflow for version tags	Success	feature/release-workflow	UmanHaroon1177
ci: add automated release workflow for version tags	Success	feature/release-workflow	UmanHaroon1177
Feature/prediction api	Success	feature/prediction-api	UmanHaroon1177
fix correct health endpoint test	Success	feature/prediction-api	UmanHaroon1177

Reference to dedicated workflow created.

Part 4 — Pull Request Requirements

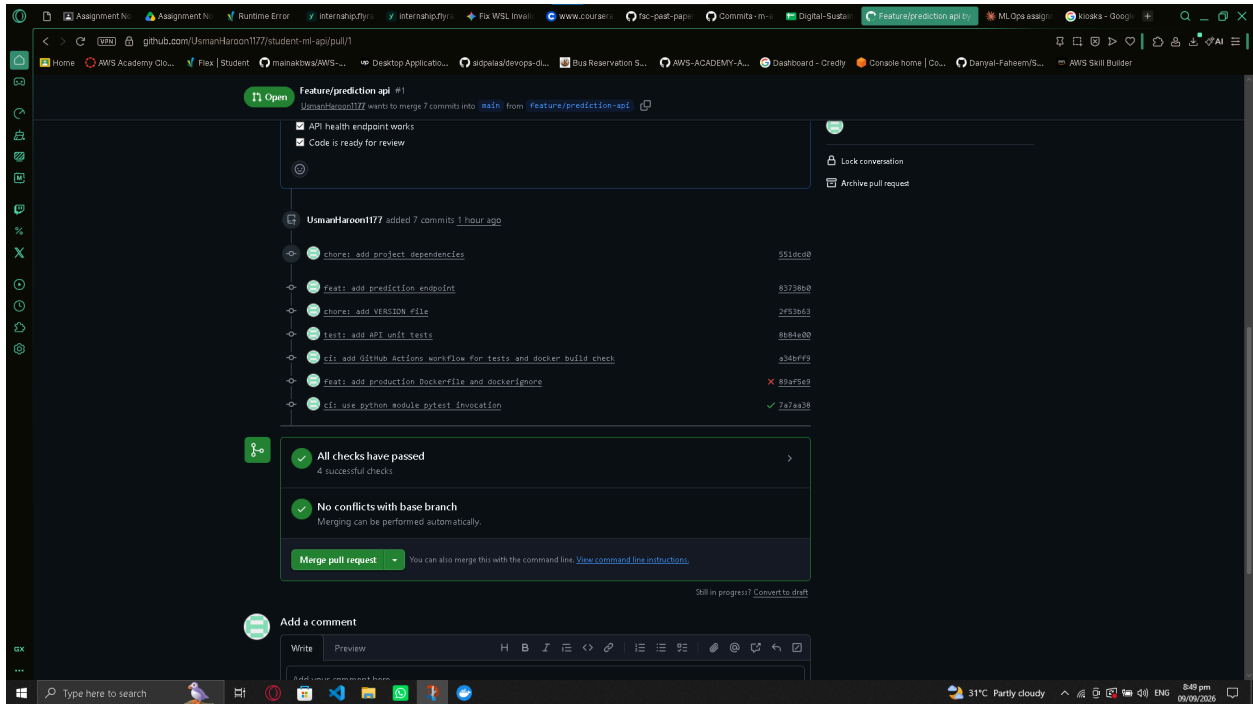
PR #1 opened



The pull request page with the full description filled in and checks just starting. This is the "before" state, taken right after opening the PR.

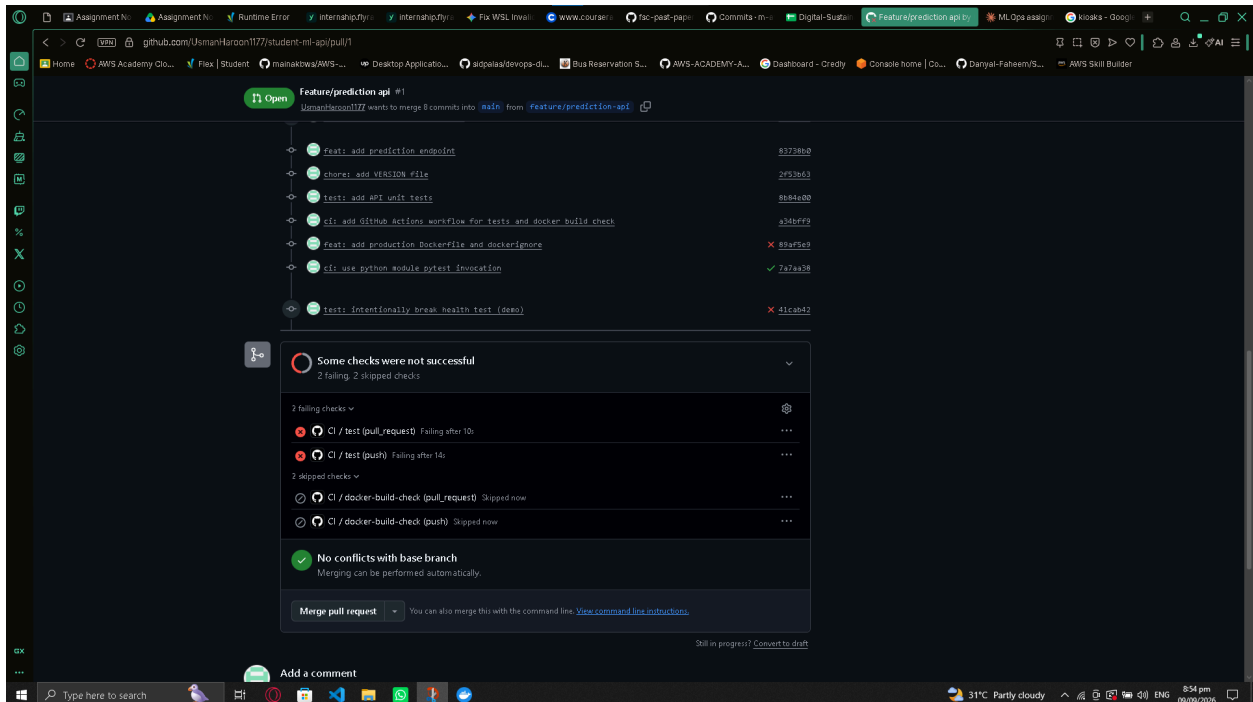
Part 5 — GitHub Actions CI

CI passing on PR #1



Same PR page, now showing all checks passed in green. Shows the pipeline ran successfully before any merge happened.

Part 6 — Introduce a Deliberate Failure CI failing after the broken test





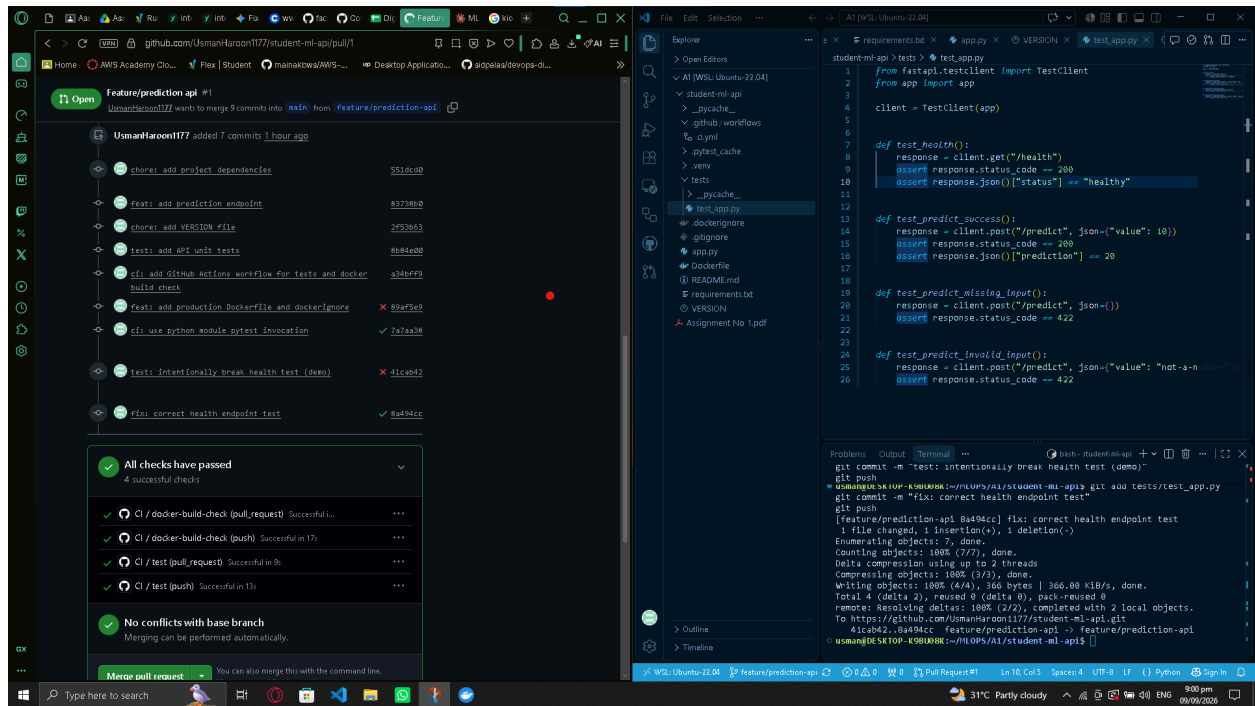
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a file named `test_app.py` selected. The code editor shows the following Python code:

```
1 from fastapi.testclient import TestClient
2 from app import app
3
4 client = TestClient(app)
5
6
7 def test_health():
8     response = client.get("/health")
9     assert response.status_code == 200
10    assert response.json()["status"] == "wrong"
11
12
13 def test_predict_success():
14     response = client.post("/predict", json={"value": 10})
15     assert response.status_code == 200
16     assert response.json()["prediction"] == 20
17
18
19 def test_predict_missing_input():
20     response = client.post("/predict", json={})
21     assert response.status_code == 422
22
23
24 def test_predict_invalid_input():
25     response = client.post("/predict", json={"value": "not-a-number"})
26     assert response.status_code == 422
```

The assertion on line 10, `assert response.json()["status"] == "wrong"`, is highlighted in blue, indicating it has failed. A red X icon is visible in the top right corner of the code editor, indicating a failed test run.

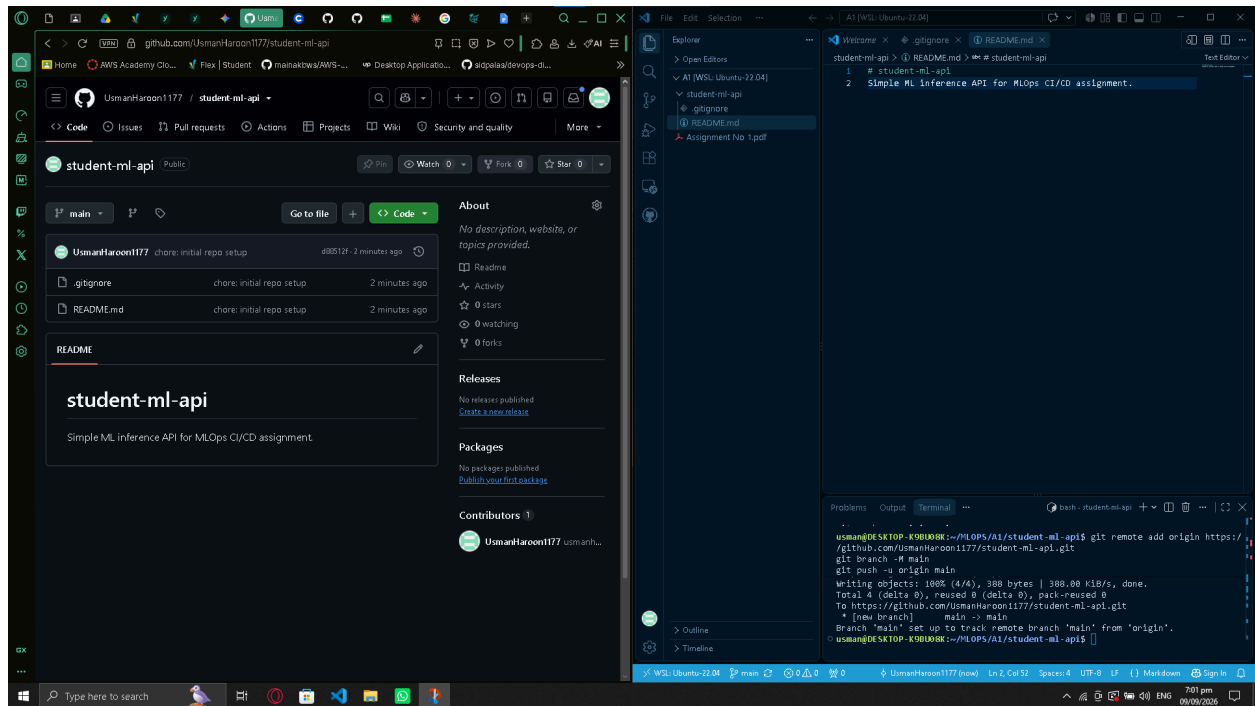
The PR showing a red X after an intentionally broken assertion was pushed. This is the required evidence that CI can actually catch failures. Actual failure log has been shown.

CI back to passing



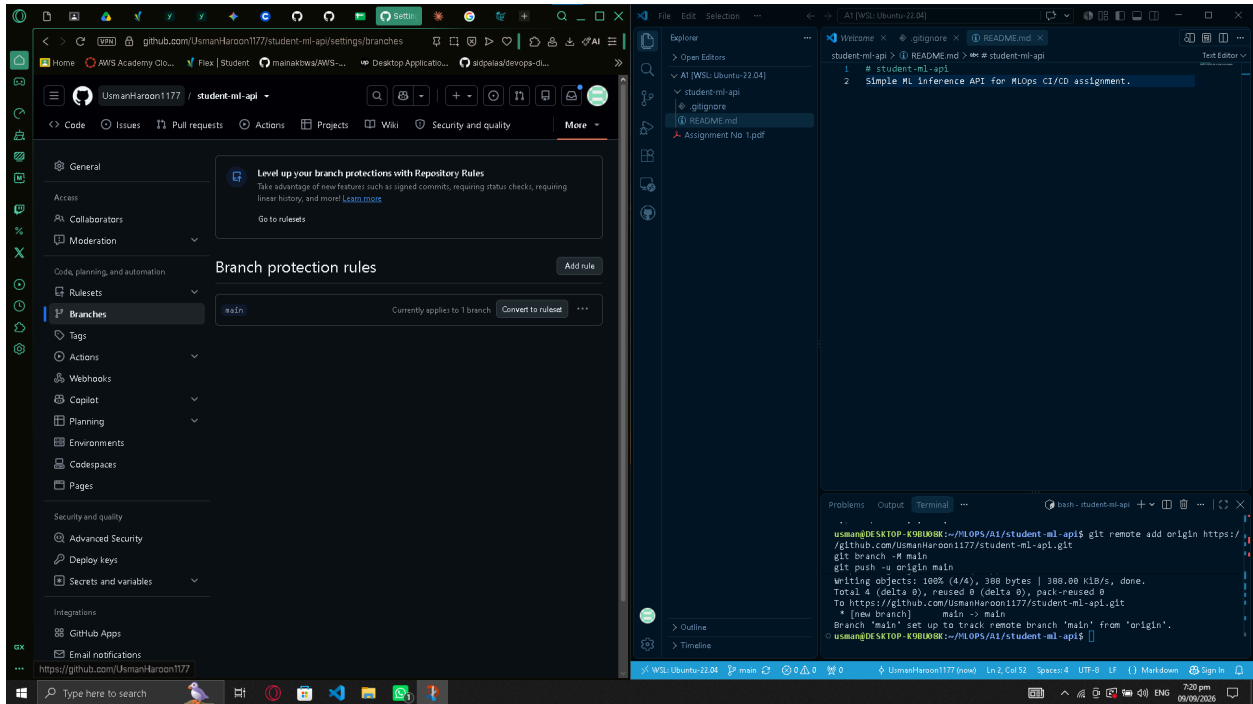
The same PR after pushing the fix commit, checks green again.
Completes the fail-then-fix story required for this part.

Part 7 — Protect the Main Branch Repo created on GitHub



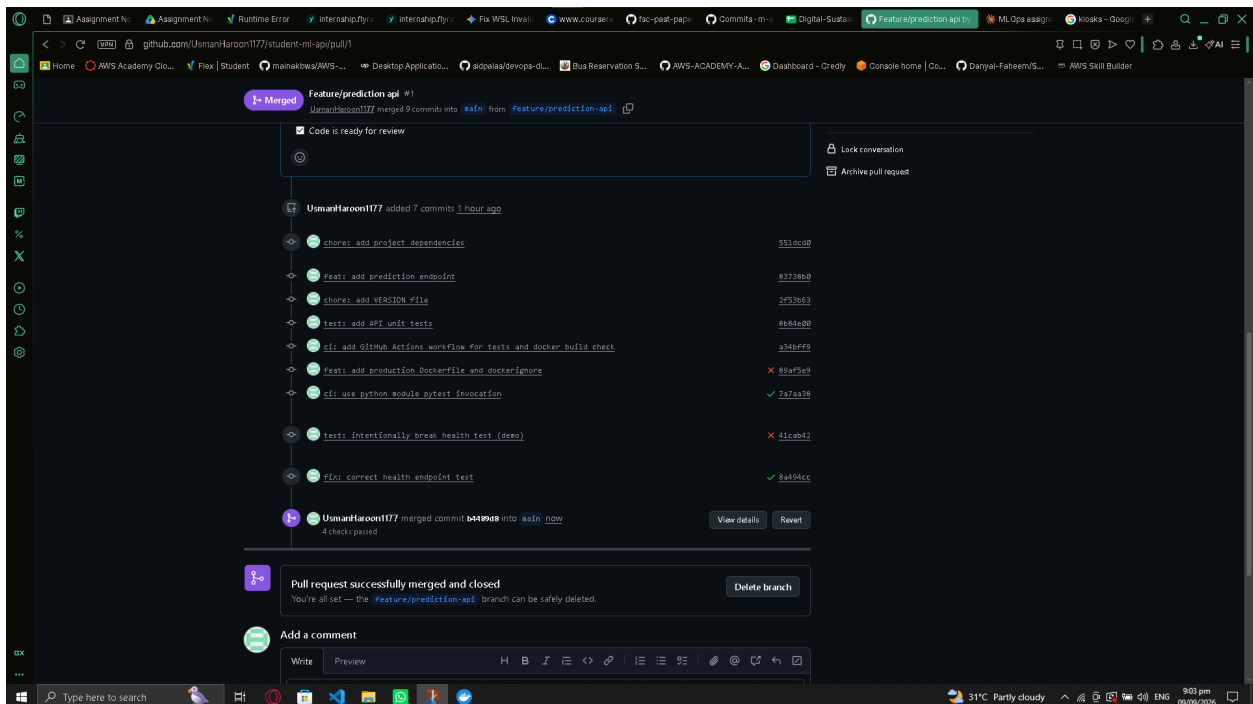
Shows the repository right after the first push, with the initial commit visible. Confirms the project actually exists on GitHub and isn't just local.

Branch protection rule for main



The Branches page showing the saved protection rule. Proves pull requests and passing checks are required before merging into main.

Part 8 — Merge the Pull Request PR #1 merged(Squash and merge)

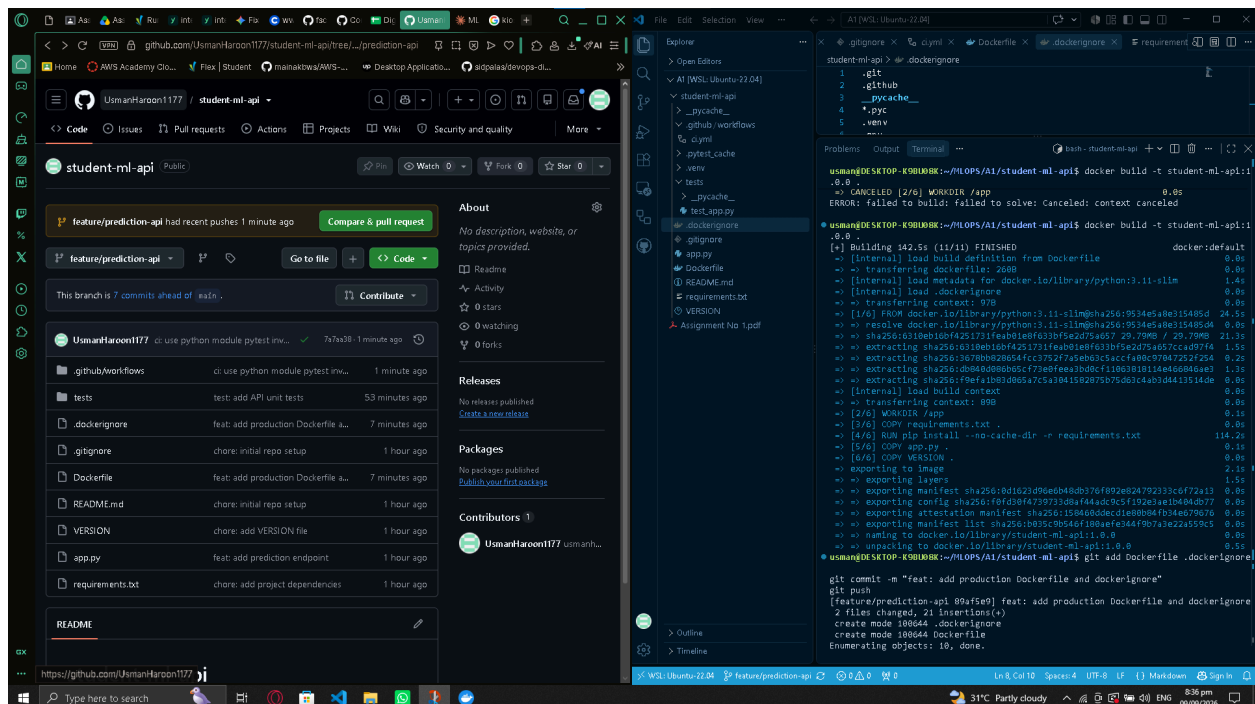


Why?

Squash merge keeps main's history linear and readable, each feature becomes one clean commit on main, while the exploratory/fix commits stay visible inside the PR itself.

Part 9 — Dockerize the Application

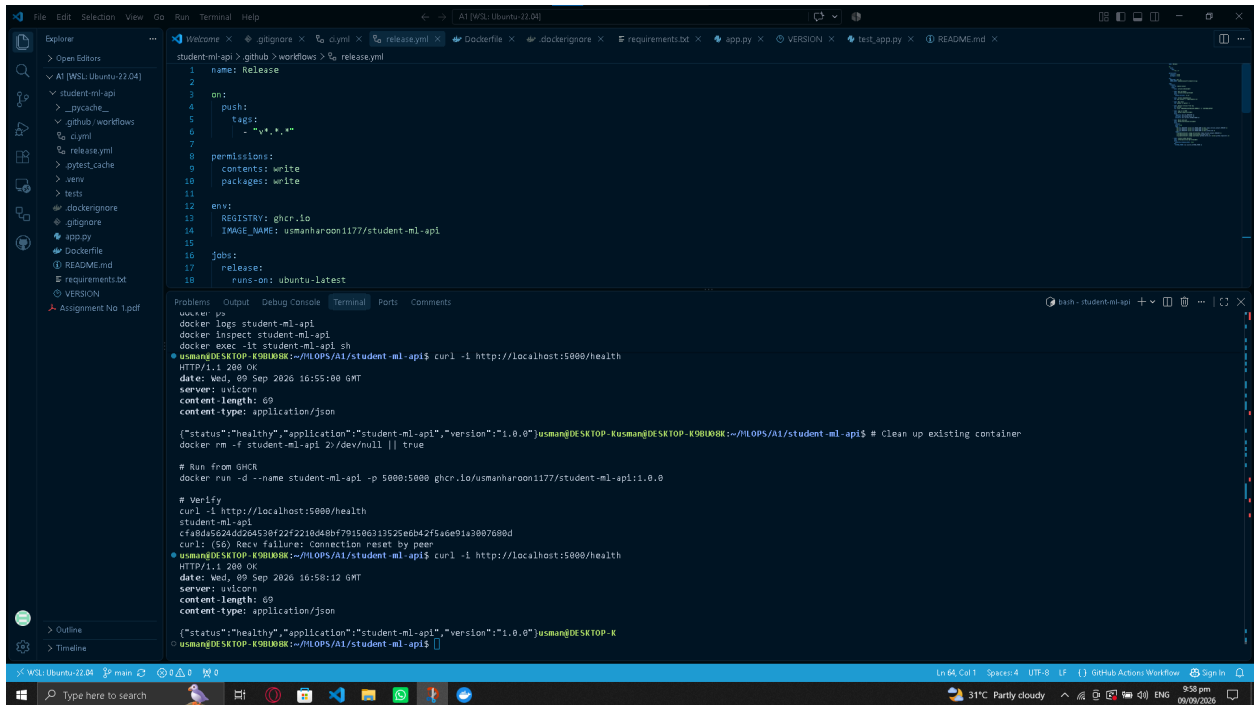
Docker build succeeds locally



Full terminal output of the docker build command finishing successfully. Confirms the Dockerfile works correctly before it's ever tested in CI.

Part 10 — Build the Docker Image Locally

Health check on the running container



The screenshot shows a VS Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure with files like `release.yml`, `Dockerfile`, `requirements.txt`, `app.py`, `VERSION`, `test_app.py`, and `README.md`. The `release.yml` file is open in the editor, showing a GitHub Actions workflow for building and pushing a Docker image. The terminal window shows the output of a `curl` command hitting the `/health` endpoint of a running container. The output is a JSON response indicating the container is healthy.

```
1 name: Release
2
3 on:
4   push:
5     tags:
6       - "v*.*.*"
7
8 permissions:
9   contents: write
10  packages: write
11
12 env:
13   REGISTRY: ghcr.io
14   IMAGE_NAME: usmanharoon1177/student-ml-api
15
16 jobs:
17   release:
18     runs-on: ubuntu-latest
```

```
usman@DESKTOP-K9B1B8K:~/HOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
HTTP/1.1 200 OK
date: Wed, 09 Sep 2020 16:55:00 GMT
server: uvicorn
content-length: 69
content-type: application/json

{"status": "healthy", "application": "student-ml-api", "version": "1.0.0"}
usman@DESKTOP-K9B1B8K:~/HOPS/AI/student-ml-api$ # Clean up existing container
docker rm -f student-ml-api 2>/dev/null || true
# Run from GHCR
docker run -d --name student-ml-api -p 5000:5000 ghcr.io/usmanharoon1177/student-ml-api:1.0.0
# Verify
curl -i http://localhost:5000/health
student-ml-api
cf4d452a4d2d630f2f2210d40bf791506212525e6b42f5ade91a3007690d
curl: (50) Recv failure: Connection reset by peer
usman@DESKTOP-K9B1B8K:~/HOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
HTTP/1.1 200 OK
date: Wed, 09 Sep 2020 16:58:12 GMT
server: uvicorn
content-length: 69
content-type: application/json

{"status": "healthy", "application": "student-ml-api", "version": "1.0.0"}
usman@DESKTOP-K9B1B8K:~/HOPS/AI/student-ml-api$
```

Curl output hitting the running container's `/health` endpoint.
Confirms the pulled image actually runs and responds correctly.

Part 11 — Docker Image Inspection

Docker inspection commands

docker images

The screenshot shows a VS Code editor with a Dockerfile open. The Dockerfile defines a container named 'student-ml-api' based on 'ghcr.io/usmanharoon1177/student-ml-api:1.0.0'. It includes a 'RELEASE' label, 'on-push' triggers, and 'permissions' for writing contents and packages. The container is run with 'runs-on: ubuntu-latest'.

```
1 name: Release
2
3 on:
4   push:
5     tags:
6       - "v*.*.*"
7
8 permissions:
9   contents: write
10  packages: write
11
12 env:
13   REGISTRY: ghcr.io
14   IMAGE_NAME: usmanharoon1177/student-ml-api
15
16 jobs:
17   release:
18     runs-on: ubuntu-latest
```

The terminal output shows the following commands and results:

```
usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
content-length: 69
content-type: application/json

{"status":"healthy","application":"student-ml-api","version":"1.0.0"}usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ # Clean up existing container
docker rm -f student-ml-api 2/dev/null || true

# Run from GHCR
docker run -d --name student-ml-api -p 5000:5000 ghcr.io/usmanharoon1177/student-ml-api:1.0.0

# Verify
curl -i http://localhost:5000/health
student-ml-api
cf8bd5624d62d4539f22f2218d40b7791506312525ed42f5a6e91a380760bd
curl: (56) Recv failure: Connection reset by peer
usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
HTTP/1.1 200 OK
date: Wed, 09 Sep 2026 16:58:12 GMT
server: uvicorn
content-length: 69
content-type: application/json

{"status":"healthy","application":"student-ml-api","version":"1.0.0"}usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ docker images
```

IMAGE	ID	DISK USAGE	Info	In Use
ghcr.io/usmanharoon1177/student-ml-api:1.0.0	13699cd7f815	214MB	20.4MB	✓
student-ml-api:1.0.0	b035c9b546f1	244MB	59.2MB	

Docker ps

The screenshot shows a VS Code editor with a Dockerfile open. The Dockerfile defines a container named 'student-ml-api' based on 'ghcr.io/usmanharoon1177/student-ml-api:1.0.0'. It includes a 'RELEASE' label, 'on-push' triggers, and 'permissions' for writing contents and packages. The container is run with 'runs-on: ubuntu-latest'.

```
1 name: Release
2
3 on:
4   push:
5     tags:
6       - "v*.*.*"
7
8 permissions:
9   contents: write
10  packages: write
11
12 env:
13   REGISTRY: ghcr.io
14   IMAGE_NAME: usmanharoon1177/student-ml-api
15
16 jobs:
17   release:
18     runs-on: ubuntu-latest
```

The terminal output shows the following commands and results:

```
usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
content-length: 69
content-type: application/json

{"status":"healthy","application":"student-ml-api","version":"1.0.0"}usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ # Clean up existing container
docker rm -f student-ml-api 2/dev/null || true

# Run from GHCR
docker run -d --name student-ml-api -p 5000:5000 ghcr.io/usmanharoon1177/student-ml-api:1.0.0

# Verify
curl -i http://localhost:5000/health
student-ml-api
cf8bd5624d62d4539f22f2218d40b7791506312525ed42f5a6e91a380760bd
curl: (56) Recv failure: Connection reset by peer
usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
HTTP/1.1 200 OK
date: Wed, 09 Sep 2026 16:58:12 GMT
server: uvicorn
content-length: 69
content-type: application/json

{"status":"healthy","application":"student-ml-api","version":"1.0.0"}usman@DESKTOP-K9BUB8K:~/PLOPS/AI/student-ml-api$ docker images
```

IMAGE	ID	DISK USAGE	Info	In Use
ghcr.io/usmanharoon1177/student-ml-api:1.0.0	13699cd7f815	214MB	20.4MB	✓
student-ml-api:1.0.0	b035c9b546f1	244MB	59.2MB	

The terminal also shows the output of 'docker ps' command, which lists the running container 'student-ml-api' with its ID, image, command, status, ports, and name.

docker logs student-ml-api & docker inspect student-ml-api

The screenshot shows a Linux terminal window with a dark theme. The top bar displays the system status: File Edit Selection View Go Run Terminal Help, along with window management icons and system information (31°C, 100% battery, 09/09/2023).

The terminal content is as follows:

```
usman@DESKTOP-K9H0K8W:~/PLOPS/AI/student-ml-api$ curl -i http://localhost:5000/health
{"status": "Healthy", "application": "student-ml-api", "version": "1.0.0"}
usman@DESKTOP-K9H0K8W:~/PLOPS/AI/student-ml-api$ docker images
REPOSITORY              ID                                DISK USAGE    Info  In Use
ghcr.io/usmanharoon177/student-ml-api:1.0.0 1369dc7f815    214MB         20.4MB
student-ml-api:1.0.0      b935c9b546f1    244MB         59.2MB

usman@DESKTOP-K9H0K8W:~/PLOPS/AI/student-ml-api$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED   STATUS    PORTS                               NAMES
cfa8d45624d   ghcr.io/usmanharoon177/student-ml-api:1.0.0  "uvicorn app:app -h..." 5 minutes ago Up 5 minutes 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp student-ml-api

usman@DESKTOP-K9H0K8W:~/PLOPS/AI/student-ml-api$ docker logs student-ml-api
INFO: Started service process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
INFO: 172.17.0.1:50008 - "GET /health HTTP/1.1" 200 OK

usman@DESKTOP-K9H0K8W:~/PLOPS/AI/student-ml-api$ docker inspect student-ml-api
[
  {
    "Id": "cfa8d45624d264530f22f2210d48b0f79150613525eeb42f5a6e91a3007680d",
    "Created": "2023-09-09T16:57:00.269049323Z",
    "Path": "/bin/sh -c 'uvicorn'",
    "Args": [
      "app:app",
      "-host",
      "0.0.0.0",
      "-port",
      "5000"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 884,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2023-09-09T16:57:00.568808802Z",
      "FinishedAt": "2023-09-09T16:58:08.882Z"
    },
    "Image": "sha256:1369dc7f815b6453c31bb78d6c77d933c03441b2077c73451c18c9644",
    "ResolvConfPath": "/var/lib/docker/containers/cfa8d45624d264530f22f2210d48b0f79150613525eeb42f5a6e91a3007680d/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/cfa8d45624d264530f22f2210d48b0f79150613525eeb42f5a6e91a3007680d/hostname",
    "HostsPath": "/var/lib/docker/containers/cfa8d45624d264530f22f2210d48b0f79150613525eeb42f5a6e91a3007680d/hosts",
    "LogPath": "/var/lib/docker/containers/cfa8d45624d264530f22f2210d48b0f79150613525eeb42f5a6e91a3007680d/log",
    "Name": "/student-ml-api",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": ""
  }
]
```

Open Editors

A1 [WSL: Ubuntu-22.04]

student-ml-api

> __pycache__

> .github / workflows

ci.yml

release.yml

> .pytest_cache

> .venv

> tests

.dockerignore

.gitignore

app.py

Dockerfile

README.md

requirements.txt

VERSION

Assignment No 1.pdf

> Outline

> Timeline

usman@DESKTOP-K9BU08K:~/MLOPS/A1/student-ml-api\$ docker inspect student-ml-api

```
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 904,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-09-09T16:57:00.568008822Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:13609cd7f815365453cc3518bc78d68c77d833ac0344cb2d77cf3451c18c9b44",
    "ResolveConfPath": "/var/lib/docker/containers/cfa8da5624dd264530f22f2210d48bf791506313525e6b42f5a6e91a3007680d/resolve.conf",
    "HostnamePath": "/var/lib/docker/containers/cfa8da5624dd264530f22f2210d48bf791506313525e6b42f5a6e91a3007680d/hostname",
    "HostsPath": "/var/lib/docker/containers/cfa8da5624dd264530f22f2210d48bf791506313525e6b42f5a6e91a3007680d/hosts",
    "LogPath": "/var/lib/docker/containers/cfa8da5624dd264530f22f2210d48bf791506313525e6b42f5a6e91a3007680d/cfa8da5624dd264530f22f2210d48bf791506313525e6b42f5a6e91a3007680d-json.log",
    "Name": "/student-ml-api",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "bridge",
      "PortBindings": {
        "5000/tcp": [
          {
            "HostIp": "",
            "HostPort": "5000"
          }
        ]
      },
      "RestartPolicy": {
        "Name": "no",
        "MaximumRetryCount": 0
      },
      "AutoRemove": false,
      "VolumeDriver": "",
      "VolumesFrom": null,
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Docker Sign In

31°C Partly cloudy 10:11 pm 09/09/2026

Open Editors

A1 [WSL: Ubuntu-22.04]

student-ml-api

> __pycache__

> .github/workflows

ci.yml

release.yml

pytest_cache

.venv

tests

.dockerignore

.gitignore

app.py

Dockerfile

README.md

requirements.txt

VERSION

Assignment No 1.pdf

Outline

Timeline

```
usman@DESKTOP-K9BU08K:~/MLOPS/A1/student-ml-api$ docker inspect student-ml-api
3007680d-json.log",
  "Name": "/student-ml-api",
  "RestartCount": 0,
  "Driver": "overlayfs",
  "Platform": "linux",
  "MountLabel": "",
  "ProcessLabel": "",
  "AppArmorProfile": "",
  "ExecIDs": null,
  "HostConfig": {
    "Binds": null,
    "ContainerIDFile": "",
    "LogConfig": {
      "Type": "json-file",
      "Config": {}
    },
    "NetworkMode": "bridge",
    "PortBindings": {
      "5000/tcp": [
        {
          "HostIp": "",
          "HostPort": "5000"
        }
      ]
    },
    "RestartPolicy": {
      "Name": "no",
      "MaximumRetryCount": 0
    },
    "AutoRemove": false,
    "VolumeDriver": "",
    "VolumesFrom": null,
    "ConsoleSize": [
      29,
      84
    ],
    "CapAdd": null,
    "CapDrop": null,
    "CgroupnsMode": "private",
    "Dns": null,
    "DnsOptions": [],
    "DnsSearch": [],
    "ExtraHosts": null,
    "GroupAdd": null,
    "IpcMode": "private",
    "Cgroup": "",
    "Links": null,
    "OomScoreAdj": 0,
    "PidMode": "",
    "Privileged": false,
    "PublishAllPorts": false,
```

WSL: Ubuntu-22.04

main

Ln 1, Col 1

Spaces: 4

UTF-8

LF

{ } Docker

Sign In

31°C Partly cloudy

10:11 pm

09/09/2026

> Open Editors

✓ A1 [WSL: Ubuntu-22.04]

✓ student-ml-api

> __pycache__

✓ .github/workflows

📄 ci.yml

📄 release.yml

> .pytest_cache

> .venv

> tests

🔗 .dockerignore

🔗 .gitignore

🔗 app.py

🔗 Dockerfile

📄 README.md

📄 requirements.txt

🕒 VERSION

📄 Assignment No. 1.pdf

> Outline

> Timeline

```
usman@DESKTOP-K9BU08K:~/MLOPS/A1/student-ml-api$ docker inspect student-ml-api

[{"Id": "8c6f98aabe8", "Image": "ghcr.io/usmanharoon1177/student-ml-api:1.0.0", "ContainerName": "student-ml-api", "Labels": {}, "Mounts": [], "Config": {"Hostname": "cfa8da5624dd", "Domainname": "", "User": "", "AttachStdin": false, "AttachStdout": false, "AttachStderr": false, "ExposedPorts": {"5000/tcp": {}}, "Tty": false, "OpenStdin": false, "StdinOnce": false, "Env": ["PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin", "LANG=C.UTF-8", "GPG_KEY=A035C8C19219BA821ECEA86B64E628F8D684696D", "PYTHON_VERSION=3.11.16", "PYTHON_SHA256=91bcdebfdde239a003ae93738a7fce0f9230fee5c4bc2b86f6e6e8c6f98aabe8"], "Cmd": ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "5000"]}, "ReadonlyPaths": ["/proc/bus", "/proc/fs", "/proc/irq", "/proc/sys", "/proc/sysrq-trigger"], "Storage": {"RootFS": {"Snapshot": {"Name": "overlayfs"}}, "Mounts": []}}
```

WSL: Ubuntu-22.04

🔄 main

🔍 0 🔔 0

Ln 1, Col 1

Spaces: 4

UTF-8

LF

{ } Docker

Sign In

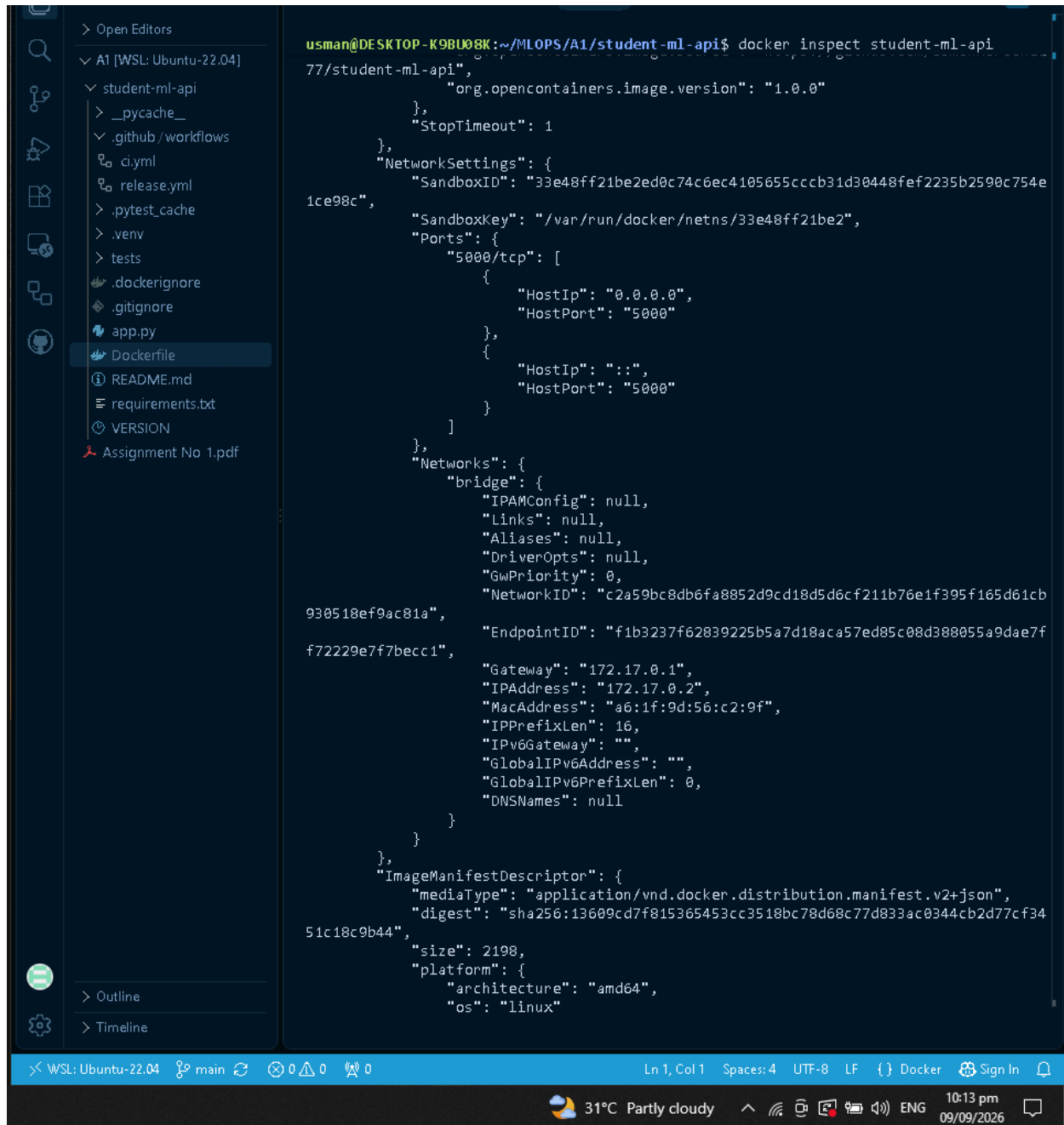
🔔

31°C Partly cloudy

10:13 pm

09/09/2026

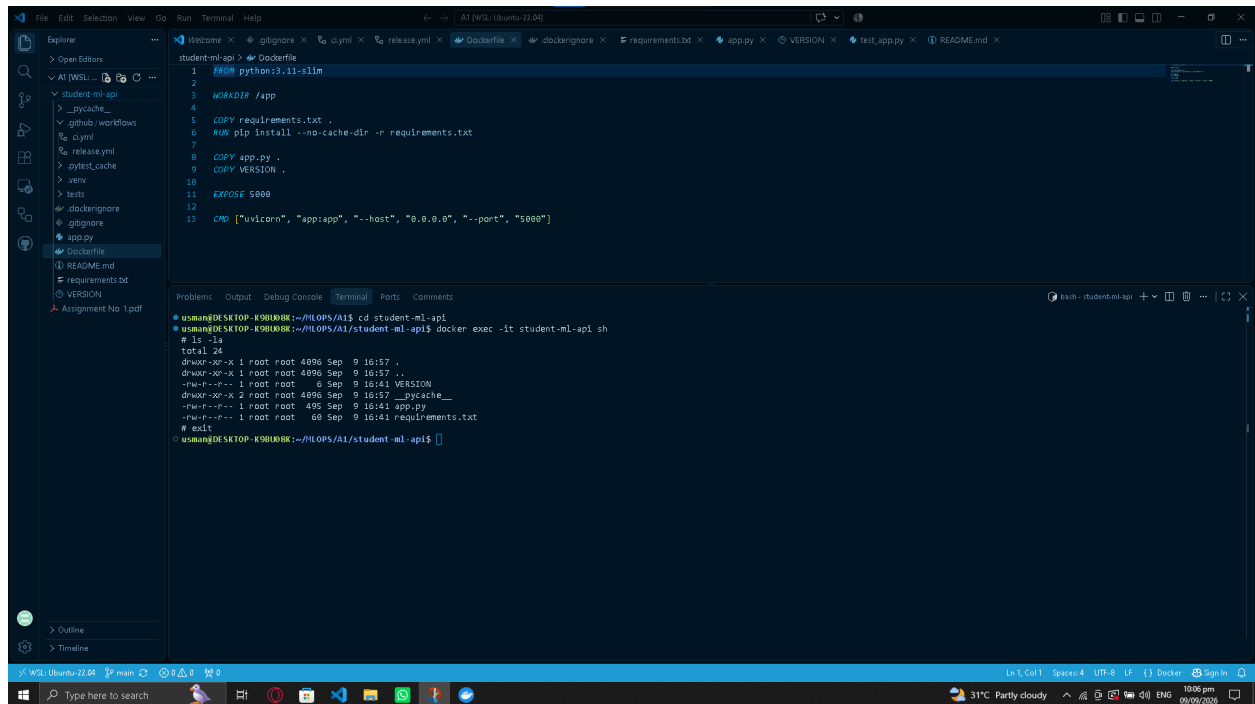
🗨



The screenshot shows a VS Code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project named 'student-ml-api' with files like 'Dockerfile', 'README.md', 'requirements.txt', and 'VERSION'. The terminal shows the command 'docker inspect student-ml-api' and its output, which is a JSON object containing details about the container and its image.

```
usman@DESKTOP-K9BU08K:~/MLOPS/A1/student-ml-api$ docker inspect student-ml-api
[{"id": "77/student-ml-api",
  "org.opencontainers.image.version": "1.0.0",
  "StopTimeout": 1,
  "NetworkSettings": {
    "SandboxID": "33e48ff21be2ed0c74c6ec4105655cccb31d30448fef2235b2590c754e1ce98c",
    "SandboxKey": "/var/run/docker/netns/33e48ff21be2",
    "Ports": {
      "5000/tcp": [
        {
          "HostIp": "0.0.0.0",
          "HostPort": "5000"
        },
        {
          "HostIp": "::",
          "HostPort": "5000"
        }
      ]
    }
  },
  "Networks": {
    "bridge": {
      "IPAMConfig": null,
      "Links": null,
      "Aliases": null,
      "DriverOpts": null,
      "GwPriority": 0,
      "NetworkID": "c2a59bc8db6fa8852d9cd18d5d6cf211b76e1f395f165d61cb930518ef9ac81a",
      "EndpointID": "f1b3237f62839225b5a7d18aca57ed85c08d388055a9dae7ff72229e7f7becc1",
      "Gateway": "172.17.0.1",
      "IPAddress": "172.17.0.2",
      "MacAddress": "a6:1f:9d:56:c2:9f",
      "IPPrefixLen": 16,
      "IPv6Gateway": "",
      "GlobalIPv6Address": "",
      "GlobalIPv6PrefixLen": 0,
      "DNSNames": null
    }
  },
  "ImageManifestDescriptor": {
    "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
    "digest": "sha256:13609cd7f815365453cc3518bc78d68c77d833ac0344cb2d77cf3451c18c9b44",
    "size": 2198,
    "platform": {
      "architecture": "amd64",
      "os": "linux"
    }
  }
}]
```

docker exec -it student-ml-api sh
ls -la



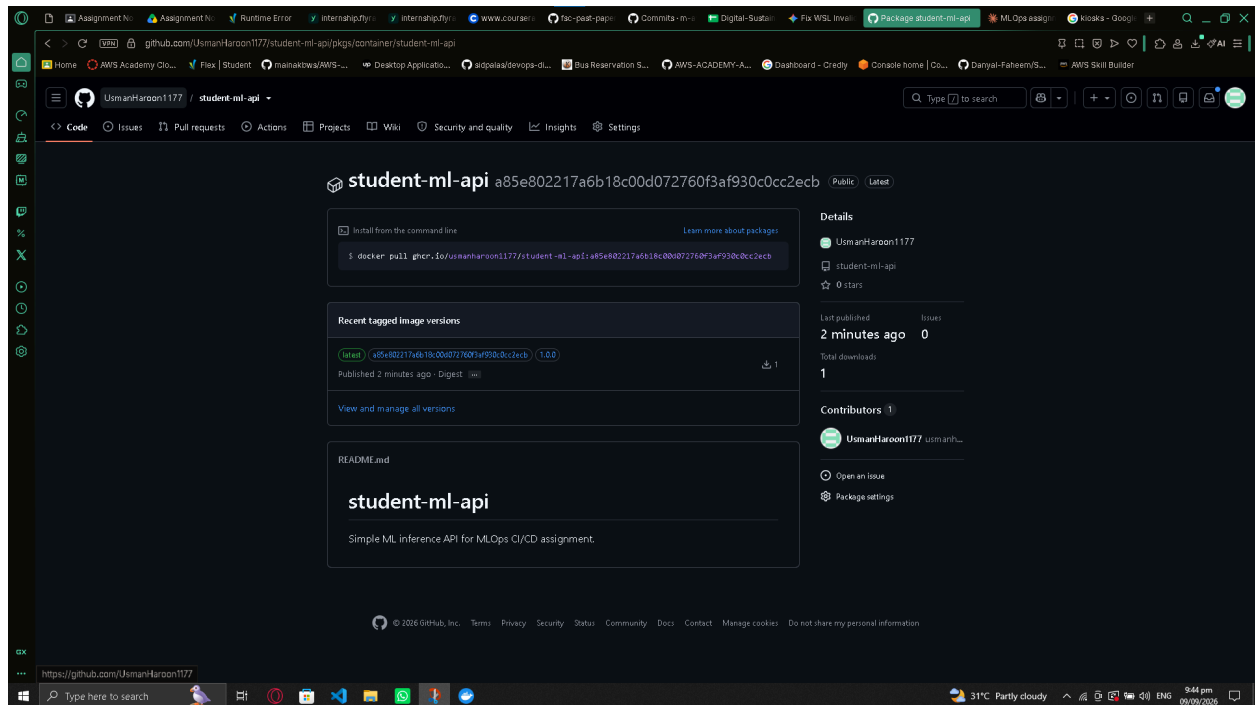
```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app.py .
9 COPY VERSION .
10
11 EXPOSE 5000
12
13 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "5000"]
```

```
usman@DESKTOP-K9B1M8K:~/MLOPS/A1$ cd student-m1-api
usman@DESKTOP-K9B1M8K:~/MLOPS/A1/student-m1-api$ docker exec -it student-m1-api sh
# ls -la
total 24
drwxr-xr-x 1 root root 4096 Sep  9 16:57 .
drwxr-xr-x 1 root root 4096 Sep  9 16:57 ..
-rw-r--r-- 1 root root   6 Sep  9 16:41 VERSION
drwxr-xr-x 2 root root 4096 Sep  9 16:57 __pycache__
-rw-r--r-- 1 root root 495 Sep  9 16:41 app.py
-rw-r--r-- 1 root root 60 Sep  9 16:41 requirements.txt
# exit
usman@DESKTOP-K9B1M8K:~/MLOPS/A1/student-m1-api$
```

One screenshot each for docker images, ps, logs, inspect, and exec.
Used to identify the container ID, image ID, exposed port, CMD, and working directory.

Part 12 — Container Registry

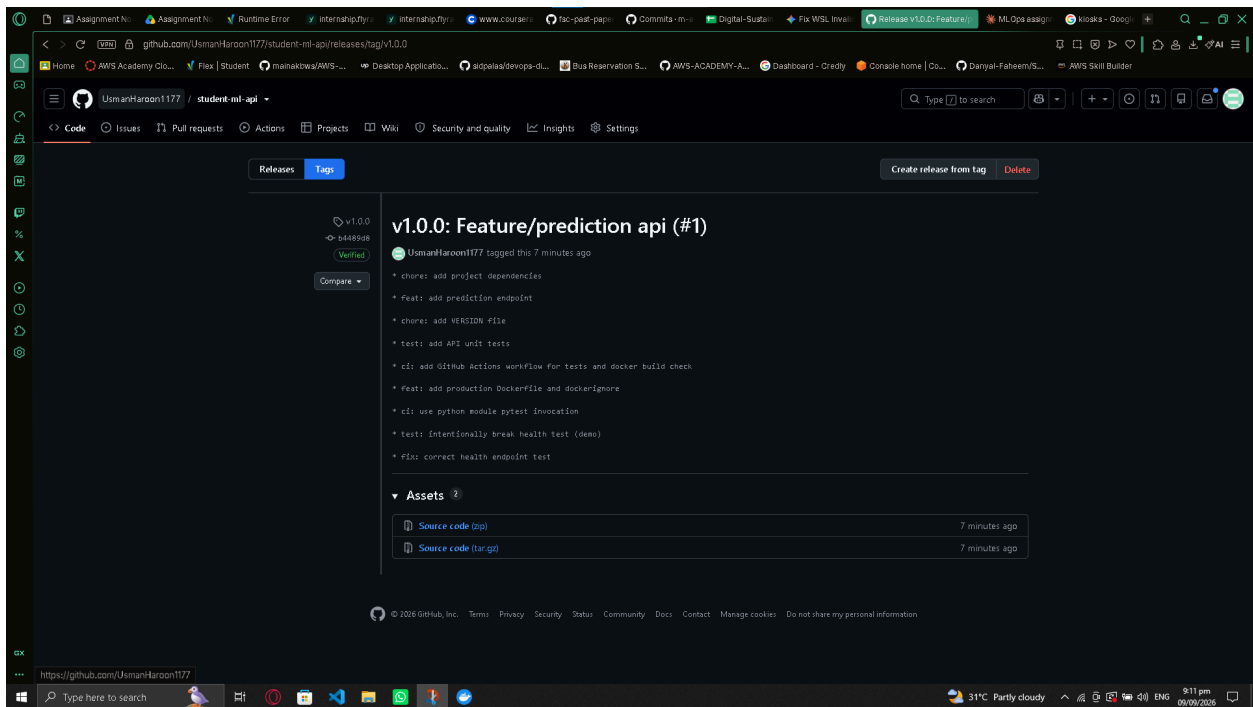
Container Registry GHCR



The Packages page shows student-ml-api with 1.0.0 and latest tags in our container Registry. The digest shown here is needed later for traceability and registry verification.

Part 13 — Git Tag and Release Version

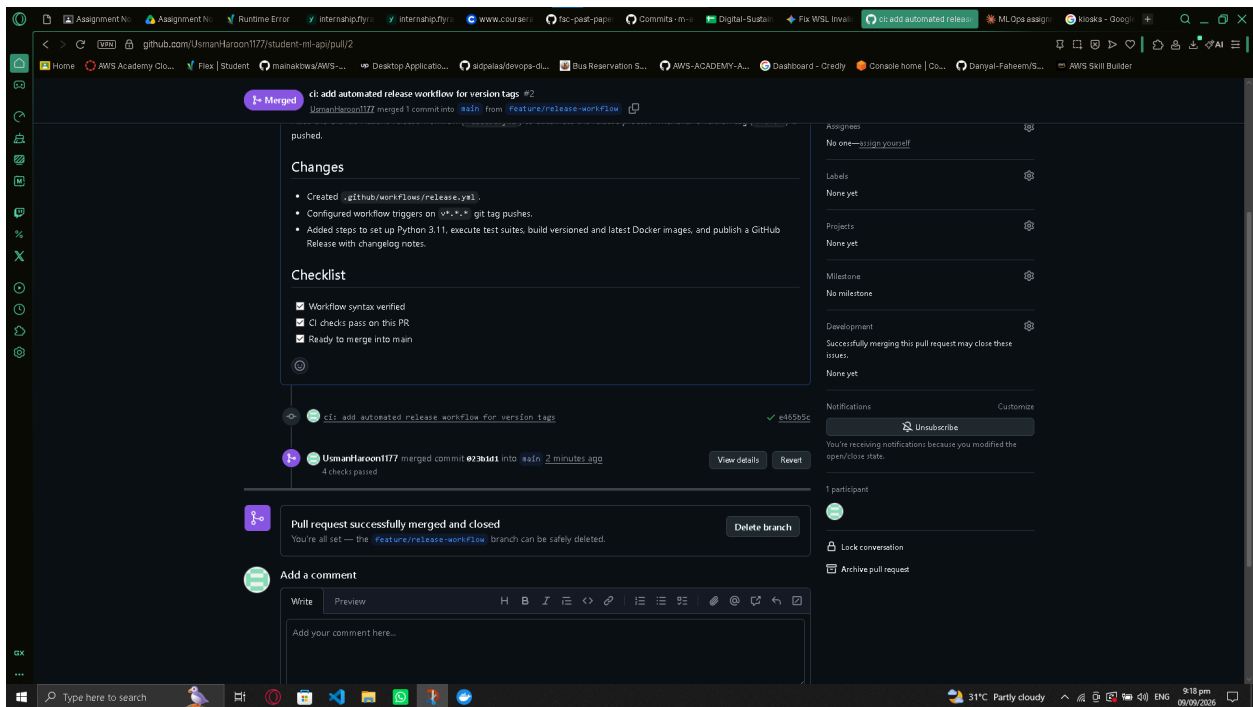
v1.0.0 tag pushed



The repository's Tags page showing v1.0.0 listed. Confirms the tag was actually pushed to GitHub, not just created locally.

Part 14 — Automated Release Workflow

PR #2 merged (release workflow added)



The merged PR that introduced release.yml into main. Second of the two required professionally documented PRs.

Part 15 — Release Pipeline Requirements

Release workflow triggered by the tag

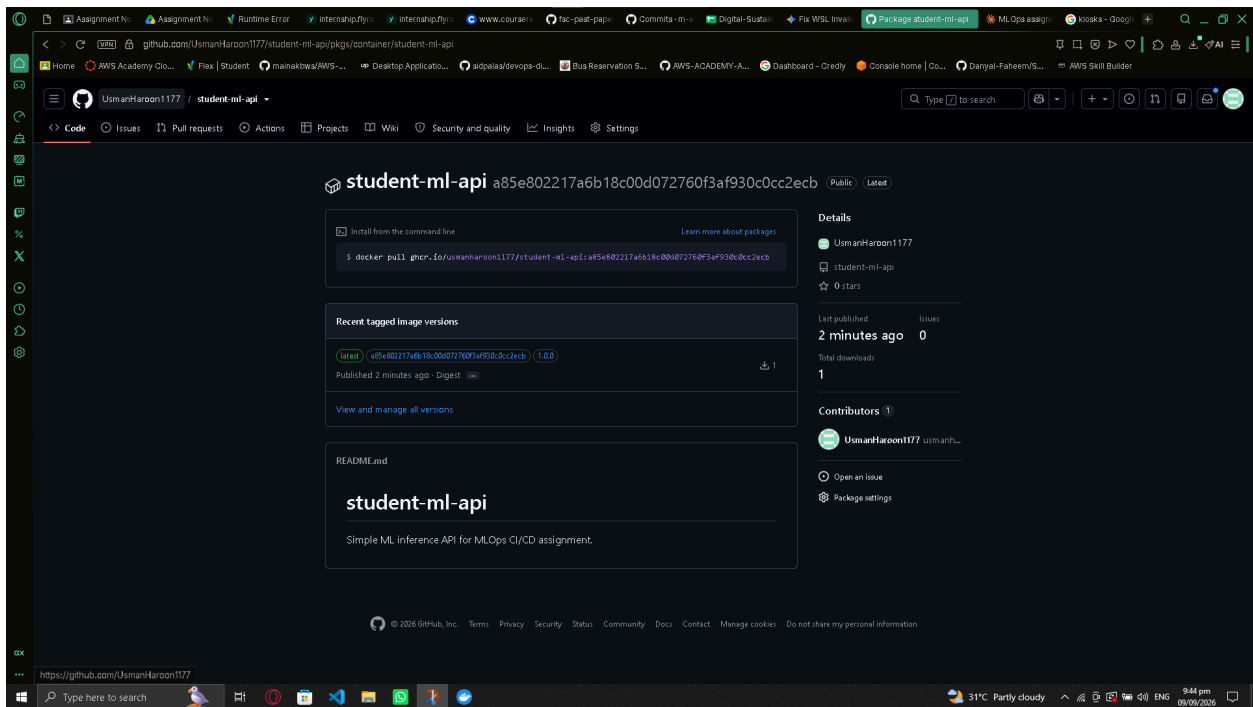
The screenshot displays the GitHub Actions interface for the repository 'UsmanHaroon1177 / student-m1-api'. The 'Actions' tab is selected, showing a workflow named 'release' with a status of 'Success'. The workflow was triggered by a push to the 'v1.0.0' tag. The summary shows a total duration of 53s and 1 artifact. Below the summary, the 'release.yml' workflow file is shown, which includes a 'release' job. The 'release summary' section provides a 'Docker Build summary' and a link to the build record. A table at the bottom lists the build records.

ID	Name	Status	Cached	Duration
UKQAMH	student-m1-api	completed	0%	12s

The Actions tab showing the release run going green after the tag push. Confirms tagging actually triggers automated publishing, as required.

Part 16 — Registry Verification

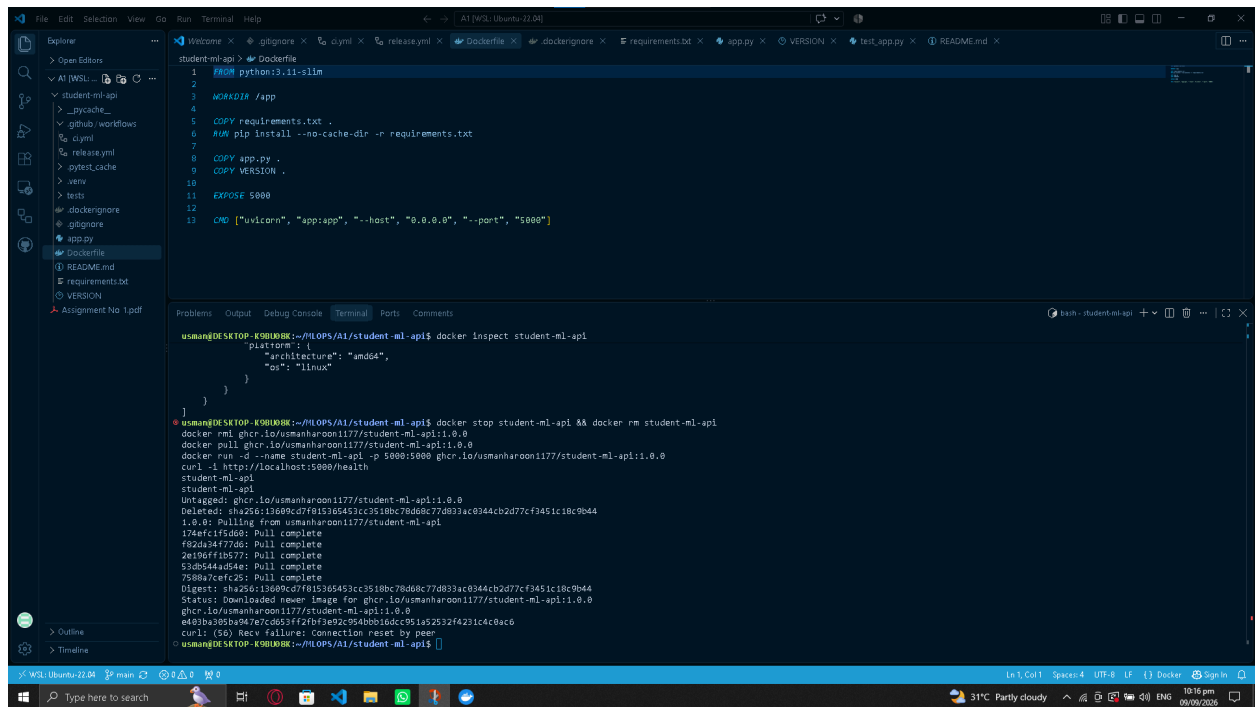
Image published to GHCR



The Packages page shows student-ml-api with 1.0.0 and latest tags. The digest shown here is needed later for traceability and registry verification.

Part 17 — Prove Artifact Reproducibility

Reproducibility: pull instead of rebuild



```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app.py .
9 COPY VERSION .
10
11 EXPOSE 5000
12
13 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "5000"]
```

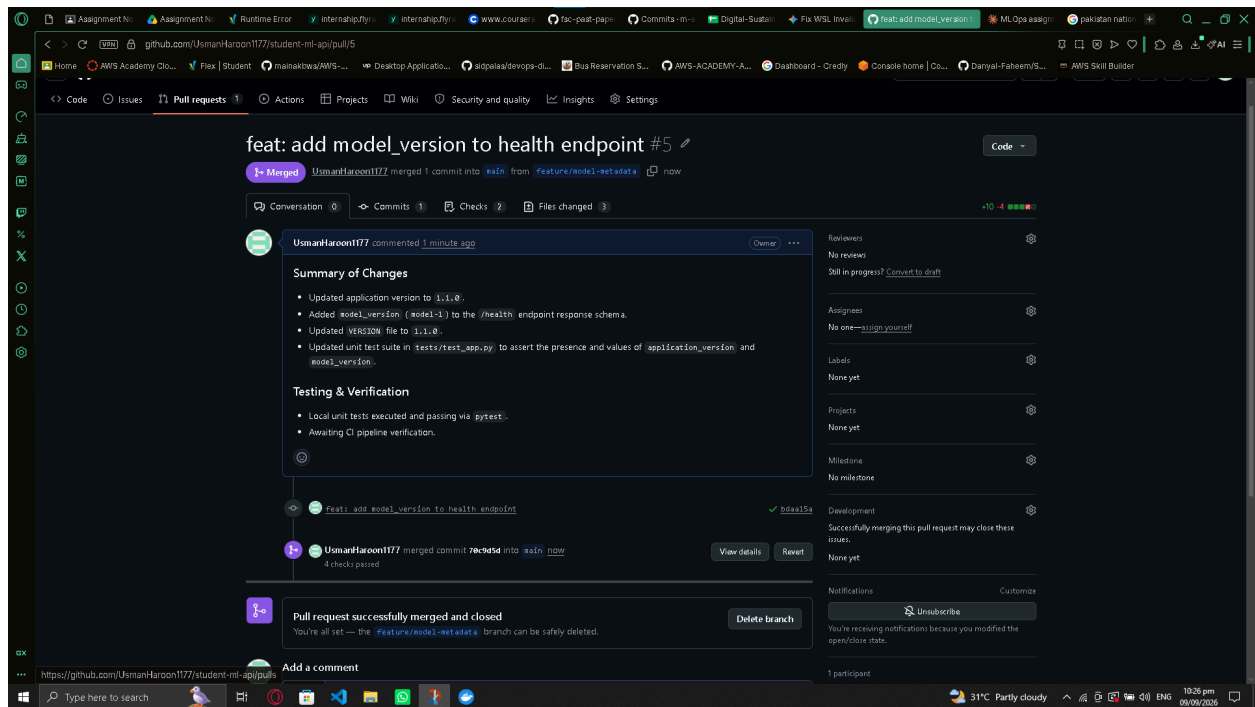
```
usman@DESKTOP-K0H0886:~/PLOPS/A1/student-ml-api$ docker inspect student-ml-api
[{"platform": "linux", "architecture": "amd64"}]
}
}

usman@DESKTOP-K0H0886:~/PLOPS/A1/student-ml-api$ docker stop student-ml-api && docker rm student-ml-api
docker rmi ghcr.io/usmanharoon1177/student-ml-api:1.0.0
docker pull ghcr.io/usmanharoon1177/student-ml-api:1.0.0
docker run -d --name student-ml-api -p 5000:5000 ghcr.io/usmanharoon1177/student-ml-api:1.0.0
curl -i http://localhost:5000/health
student-ml-api
student-ml-api
Untagged: ghcr.io/usmanharoon1177/student-ml-api:1.0.0
Deleted: sha256:13609cd7f815365453cc3518bc78d68c77d833ac0344cb2d77cf3451c18c9b44
1.0.0: Pulling from usmanharoon1177/student-ml-api
174efc155d65: Pull complete
f82da34f77d6: Pull complete
2e196ff1b577: Pull complete
53db544d54e4: Pull complete
7588a7cefc25: Pull complete
Digest: sha256:13609cd7f815365453cc3518bc78d68c77d833ac0344cb2d77cf3451c18c9b44
Status: Downloaded newer image for ghcr.io/usmanharoon1177/student-ml-api:1.0.0
ghcr.io/usmanharoon1177/student-ml-api:1.0.0
e403ba305b947e7cd653ff2fbf3e92c954bb01ddcc951a52532fa231c4c0ac0
curl: (56) Recv failure: Connection reset by peer
usman@DESKTOP-K0H0886:~/PLOPS/A1/student-ml-api$
```

The full sequence of deleting the local image, pulling from GHCR, and running it again. Proves the same artifact runs correctly without ever being rebuilt

Part 18 — Develop Version 1.1.0

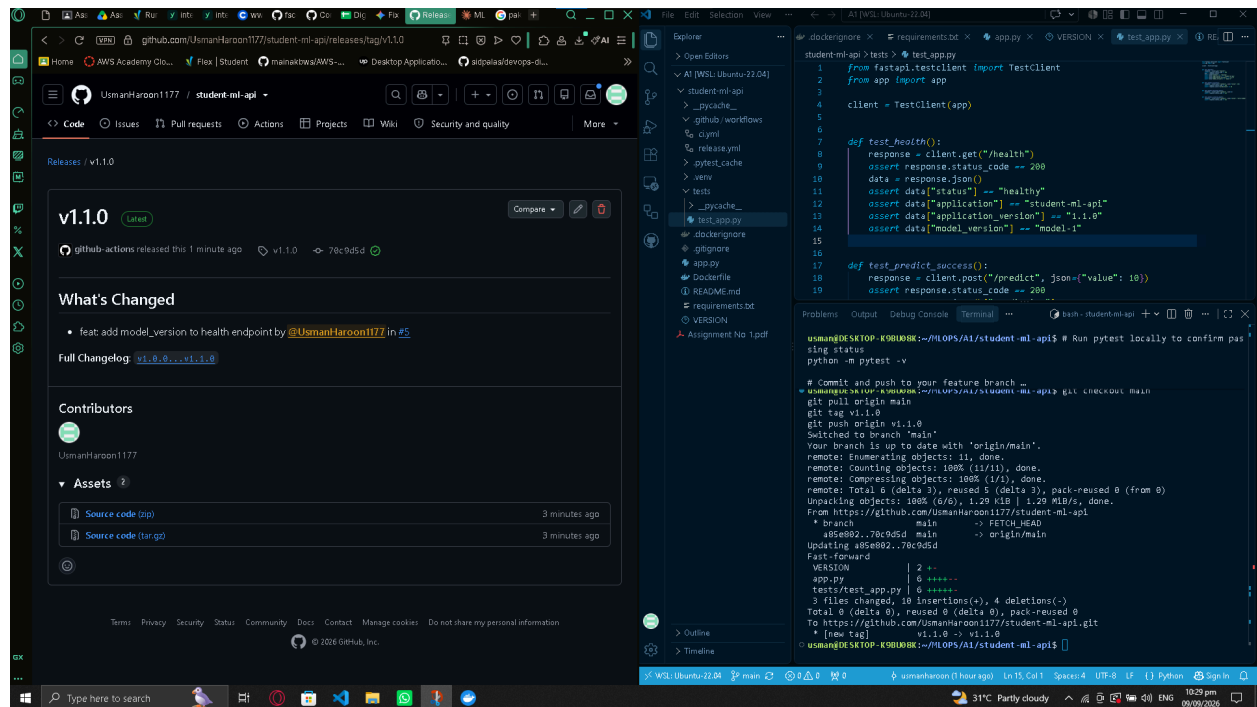
PR #5 merged (v1.1.0 feature)



The merged pull request that added `model_version` to the health endpoint. The merge commit SHA here feeds directly into the traceability table.

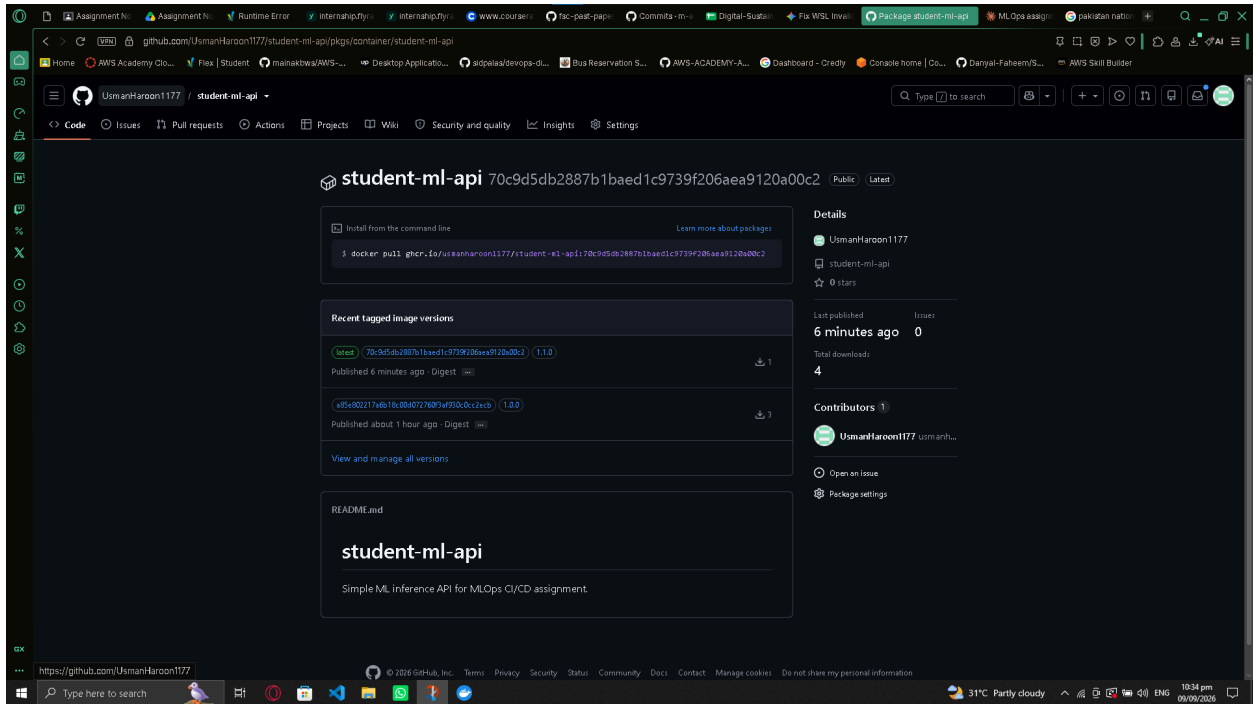
Part 19 — Release Version 1.1.0

Release workflow for v1.1.0



The Actions tab showing this second release run completing successfully. Same purpose as SS13, just for the second version.

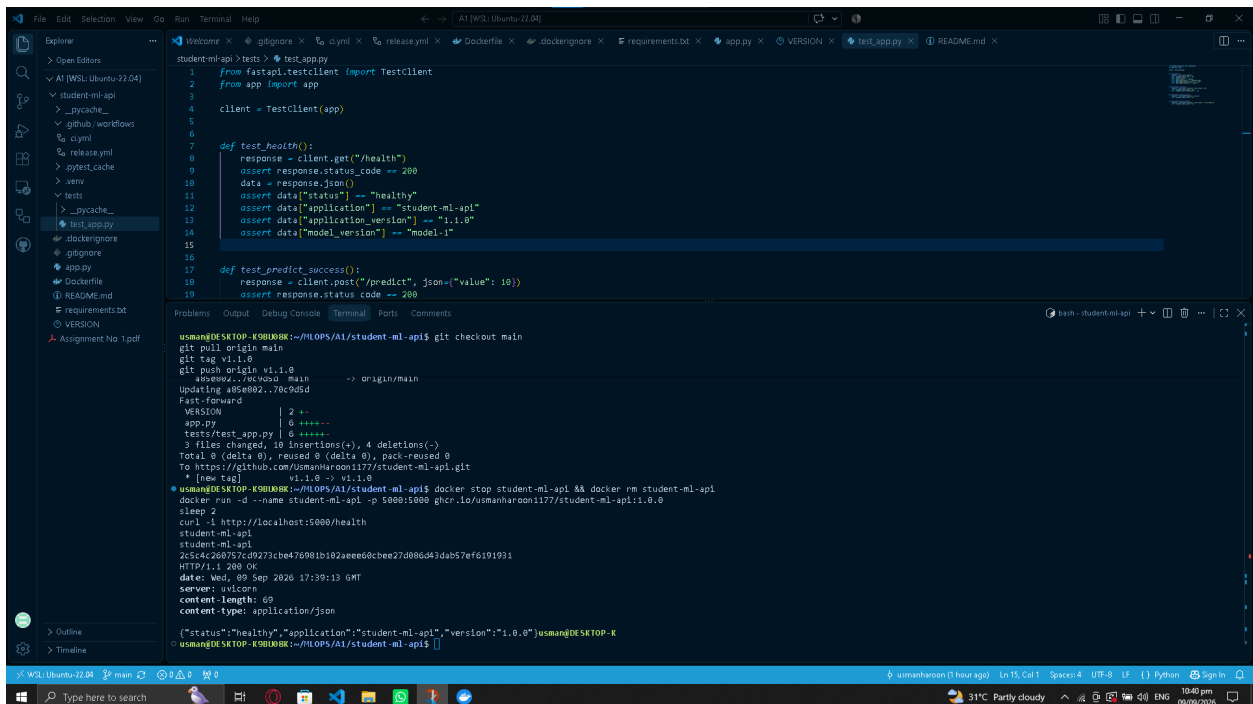
Registry now has both versions



The Packages page listing 1.0.0, 1.1.0, and latest together. Confirms latest now points to 1.1.0's digest instead of 1.0.0's.

Part 20 — Rollback Exercise

Rollback to v1.0.0



A registry rollback runs a pre-tested, immutable image in seconds with zero build steps or dependency drift. In contrast, a source rollback re-executes, installs and builds on the host, risking dependency errors and environment mismatches.

Part 21 — Traceability Challenge

Item	Value
PR Number	#5
Merge Commit SHA	70c9d5d
Git Tag	v1.1.0
Docker Image Tag	student-ml-api:1.1.0
Image Digest	sha256:84d3bdd3112b615afa4491e8ab666e41760bf8abd54ae513843f6cc090884982

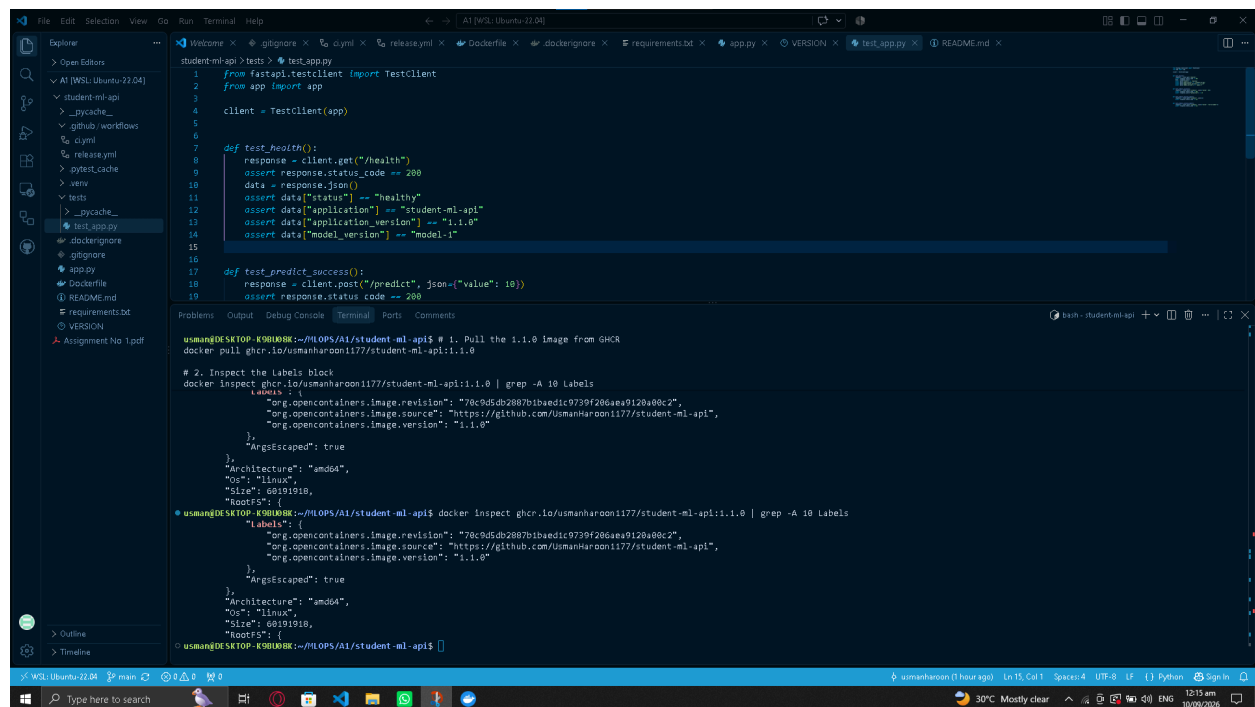
This chain shows that a single Docker image (identified by its digest) can be traced back through its version tag, to the Git tag that triggered its build, to the exact commit that was merged, to the pull request that introduced the change giving full traceability from artifact back to source.

Part 22 — Advanced GitHub Actions Requirement

`ci.yml` only validates PRs via tests and test-builds, while `release.yml` exclusively handles publishing on version tags with write permissions.

Publishing on every PR would distribute unreviewed code, pollute the registry with throwaway tags, and destroy semantic versioning by conflating test builds with stable releases. Restricting pushes to tags also limits credential access to trusted, reviewed releases.

Part 23 — OCI Labels (Image Metadata)



```
student-m1-api> tests > test_app.py
1 from fastapi.testclient import TestClient
2 from app import app
3
4 client = TestClient(app)
5
6
7 def test_health():
8     response = client.get("/health")
9     assert response.status_code == 200
10    data = response.json()
11    assert data["status"] == "healthy"
12    assert data["application"] == "student-m1-api"
13    assert data["application version"] == "1.1.0"
14    assert data["model version"] == "model-1"
15
16
17 def test_predict_success():
18     response = client.post("/predict", json={"value": 10})
19     assert response.status_code == 200
```

```
usman@DESKTOP-K5B0R0K:~/POPS/AI/student-m1-api$ # 1. Pull the 1.1.0 image from GHCR
docker pull ghcr.io/usmanharoon177/student-m1-api:1.1.0
# 2. Inspect the labels block
docker inspect ghcr.io/usmanharoon177/student-m1-api:1.1.0 | grep -A 10 Labels
Labels:
  "org.opencontainers.image.revision": "70c9d5db2087b1baed1c9739f286ae9120a00c2",
  "org.opencontainers.image.source": "https://github.com/usmanharoon177/student-m1-api",
  "org.opencontainers.image.version": "1.1.0"
}
  "ArgsEscaped": true
},
  "Architecture": "amd64",
  "Os": "linux",
  "Size": 60191918,
  "RootFS": {
    "Labels": {
      "org.opencontainers.image.revision": "70c9d5db2087b1baed1c9739f286ae9120a00c2",
      "org.opencontainers.image.source": "https://github.com/usmanharoon177/student-m1-api",
      "org.opencontainers.image.version": "1.1.0"
    },
    "ArgsEscaped": true
  },
  "Architecture": "amd64",
  "Os": "linux",
  "Size": 60191918,
  "RootFS": {
```

Terminal output showing the `org.opencontainers.image` labels on the built image. Confirms the image carries embedded metadata tracing back to its source commit and repo.

Part 24 — Commit SHA Tag

The screenshot displays the Docker Hub interface for the **student-ml-api** package. At the top, the package name is followed by the tag **580a7b4**, which is also the commit SHA. Below this, a terminal snippet shows the command to pull the image: `$ docker pull ghcr.io/usmanharoon1177/student-ml-api:580a7b4`. The 'Recent tagged image versions' section lists three versions: **latest** (tagged 580a7b4), **1.1.0**, and **70c9d5db2887b1baed1c9739f206aea9120a00c2**. Each version includes its publication time, digest, and download count. The 'Details' sidebar on the right shows the package was published 3 minutes ago, has 0 issues, and 6 total downloads. The 'Contributors' section lists UsmanHaroon1177. The README section at the bottom describes the package as a 'Simple ML inference API for ML Ops CI/CD assignment'.

student-ml-api 580a7b4 Public Latest

Install from the command line [Learn more about packages](#)

```
$ docker pull ghcr.io/usmanharoon1177/student-ml-api:580a7b4
```

Recent tagged image versions

Version	Published	Digest	Downloads
latest (580a7b4) 1.1.0	Published 3 minutes ago	sha256:68169306f403d12e19c50aaba21a0c18106332331bcb1271dca40f336594cbe	1
70c9d5db2887b1baed1c9739f206aea9120a00c2	Published about 2 hours ago		2
a85e802217a6b18c00d072760f3af930c0cc2ecb 1.0.0	Published about 3 hours ago		3

[View and manage all versions](#)

Details

UsmanHaroon1177

student-ml-api

0 stars

Last published: **3 minutes ago**

Issues: **0**

Total downloads: **6**

Contributors 1

UsmanHaroon1177

[Open an issue](#)

[Package settings](#)

README.md

student-ml-api

Simple ML inference API for ML Ops CI/CD assignment.

<https://github.com/UsmanHaroon1177/student-ml-api/pulls>

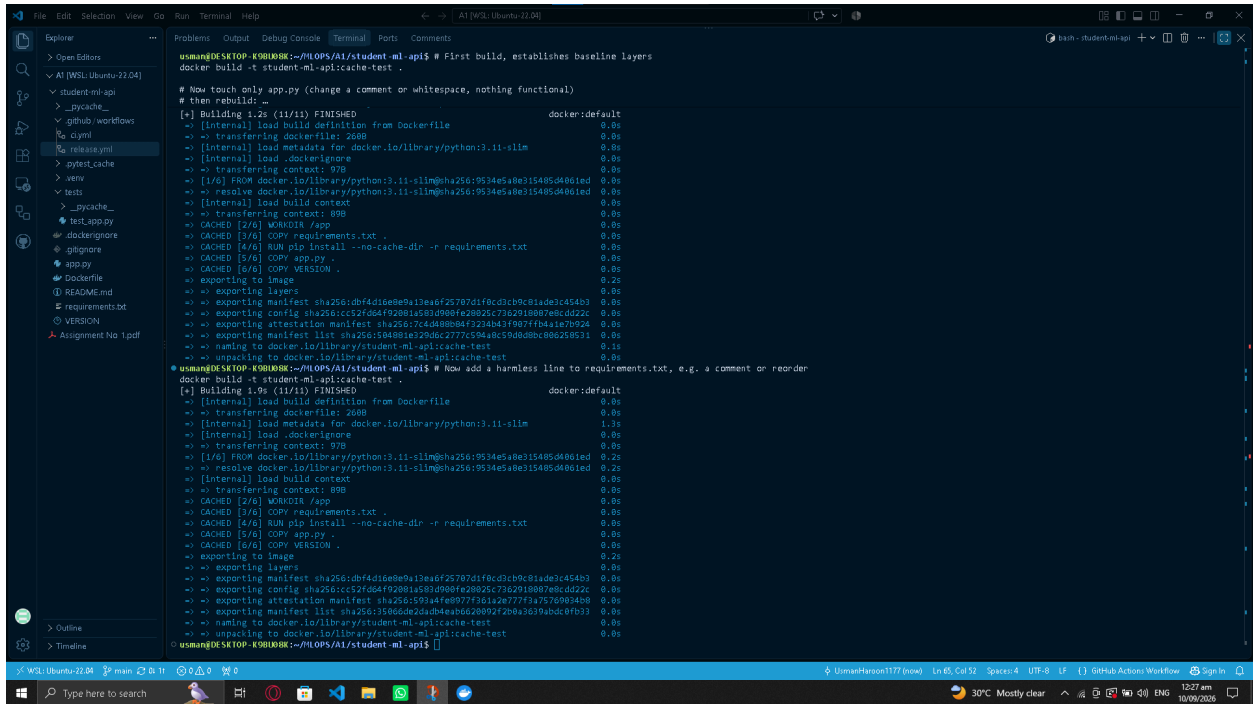
The Packages page showing a tag matching the exact commit hash, alongside 1.1.0 and latest. Confirms every release also gets a permanent, unambiguous commit-specific tag.

Every release publishes a commit-SHA tag (e.g., [student-ml-api:9c7abf1](#)), which permanently binds the image to an exact code snapshot. Unlike mutable semantic tags, it cannot be accidentally overwritten, ensuring unambiguous debugging and traceability during production incidents.

Part 25 — Docker Build Cache / Layer Ordering

[illegible]

Terminal output of the second docker build after only touching application code. Shows the dependency-install layer coming from cache instead of rerunning.



```
usman@DESKTOP-K5B108K:~/PROPS/AI/student-ml-api$ # First build, establishes baseline layers
docker build -t student-ml-api:cache-test .

# Now touch only app.py (change a comment or whitespace, nothing functional)
# then rebuild: ...
[*] Building 1.2s (11/11) FINISHED                                docker:default
-> [Internal] load build definition from Dockerfile                0.0s
-> transferring dockerfile: 268B                                   0.0s
-> [Internal] load metadata for docker.io/library/python:3.11-slim 0.0s
-> [Internal] load .dockerignore                                   0.0s
-> transferring context: 97B                                       0.0s
-> [1/6] FROM docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4861ed 0.0s
-> resolve docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4861ed 0.0s
-> [Internal] load build context                                   0.0s
-> transferring context: 89B                                       0.0s
-> CACHED [2/6] WORKDIR /app                                       0.0s
-> CACHED [3/6] COPY requirements.txt .                             0.0s
-> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [5/6] COPY app.py .                                     0.0s
-> CACHED [6/6] COPY VERSION .                                     0.0s
-> exporting to image                                              0.2s
-> exporting layers                                                0.0s
-> exporting manifest sha256:dbf4d10e0e913eaf25787d1f8d3b9c81ade3c454b7 0.0s
-> exporting config sha256:ccc2f064f92881a591d00ef20825c7362918887ebcd22c 0.0s
-> exporting attestation manifest sha256:7c4d488b084f9234b41f9077f64a1e7b924 0.0s
-> exporting manifest list sha256:584681a225d6c2777c594a6c593b8dc08625921 0.0s
-> naming to docker.io/library/student-ml-api:cache-test          0.1s
-> unpacking to docker.io/library/student-ml-api:cache-test       0.0s
usman@DESKTOP-K5B108K:~/PROPS/AI/student-ml-api$ # Now add a harmless line to requirements.txt, e.g. a comment or reorder
docker build -t student-ml-api:cache-test .
[*] Building 1.0s (11/11) FINISHED                                docker:default
-> [Internal] load build definition from Dockerfile                0.0s
-> transferring dockerfile: 268B                                   0.0s
-> [Internal] load metadata for docker.io/library/python:3.11-slim 1.1s
-> [Internal] load .dockerignore                                   0.0s
-> transferring context: 97B                                       0.0s
-> [1/6] FROM docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4861ed 0.2s
-> resolve docker.io/library/python:3.11-slim@sha256:9534e5a8e315485d4861ed 0.2s
-> [Internal] load build context                                   0.0s
-> transferring context: 89B                                       0.0s
-> CACHED [2/6] WORKDIR /app                                       0.0s
-> CACHED [3/6] COPY requirements.txt .                             0.0s
-> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt 0.0s
-> CACHED [5/6] COPY app.py .                                     0.0s
-> CACHED [6/6] COPY VERSION .                                     0.0s
-> exporting to image                                              0.2s
-> exporting layers                                                0.0s
-> exporting manifest sha256:dbf4d10e0e913eaf25787d1f8d3b9c81ade3c454b7 0.0s
-> exporting config sha256:ccc2f064f92881a591d00ef20825c7362918887ebcd22c 0.0s
-> exporting attestation manifest sha256:593A4fe077f361a2e777f347570984ab8 0.0s
-> exporting manifest list sha256:3506d62dd4e4d62089272b8a3639abdc0fb3 0.0s
-> naming to docker.io/library/student-ml-api:cache-test          0.0s
-> unpacking to docker.io/library/student-ml-api:cache-test       0.0s
usman@DESKTOP-K5B108K:~/PROPS/AI/student-ml-api$
```

Terminal output of the third docker build after modifying requirements.txt. Shows the dependency-install layer rerunning because that layer's cache was invalidated.

Comparing

Copying `requirements.txt` and running `pip install` before copying `app.py` leverages Docker's layer cache: code changes won't trigger expensive dependency reinstalls. Using `COPY . .` first would bust the cache on every trivial edit, forcing full reinstallations and significantly slowing builds.

Part 26 — Failure Analysis (pick 2, write up formally)

Port mismatch failure

The screenshot shows a VS Code editor with a Dockerfile open. The Dockerfile defines a service named 'student-m1-api' that uses the 'ghcr.io/usmanharoon1177/student-m1-api:1.1.0' image. The terminal output shows the user running 'docker build -t student-m1-api:cache-test' and then 'docker run -d --name port-test -p 8080:5000 ghcr.io/usmanharoon1177/student-m1-api:1.1.0'. The user then tests the service with 'curl -i http://localhost:5000/health', which returns a 200 OK response with a JSON body indicating the service is healthy.

```
52 FROM ghcr.io/usmanharoon1177/student-m1-api:1.1.0
53 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
54 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
55 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
56 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
57 org.opencontainers.image.version={{ steps.meta.outputs.VERSION }}
58 org.opencontainers.image.revision={{ github.sha }}
59 org.opencontainers.image.source={{ github.server_url }}/{{ github.repository }}
60
61 - name: Create GitHub Release
62   uses: softprops/action-gh-release@v2
63   with:
64     generate_release_notes: true
65   env:
66     GITHUB_TOKEN: {{ secrets.GITHUB_TOKEN }}
```

```
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$ # Now add a harmless line to requirements.txt, e.g. a comment or reorder
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$ docker build -t student-m1-api:cache-test .
[+] Building 0.0s
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$ docker run -d --name port-test -p 8080:5000 ghcr.io/usmanharoon1177/student-m1-api:1.1.0
1450b62262f2f1ab5516a0d30176249c085793bb1521c35228011155753
HTTP/1.1 200 OK
date: Wed, 09 Sep 2026 19:32:14 GMT
server: uvicorn
content-length: 69
content-type: application/json

{"status": "healthy", "application": "student-m1-api", "version": "1.0.0"}
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$ docker stop student-m1-api
student-m1-api
curl: (7) Failed to connect to localhost port 5000 after 5 ms: Connection refused
```

Fixed and working

The screenshot shows the same VS Code editor with the Dockerfile. The terminal output shows the user running 'docker stop student-m1-api' and then 'docker run -d --name port-test -p 8080:5000 ghcr.io/usmanharoon1177/student-m1-api:1.1.0'. The user then tests the service with 'curl -i http://localhost:5000/health', which returns a 200 OK response with a JSON body indicating the service is healthy.

```
52 FROM ghcr.io/usmanharoon1177/student-m1-api:1.1.0
53 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
54 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
55 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
56 LABELS={{ env.REGISTRY }}/{{ env.IMAGE_NAME }}:latest
57 org.opencontainers.image.version={{ steps.meta.outputs.VERSION }}
58 org.opencontainers.image.revision={{ github.sha }}
59 org.opencontainers.image.source={{ github.server_url }}/{{ github.repository }}
60
61 - name: Create GitHub Release
62   uses: softprops/action-gh-release@v2
63   with:
64     generate_release_notes: true
65   env:
66     GITHUB_TOKEN: {{ secrets.GITHUB_TOKEN }}
```

```
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$ docker stop student-m1-api
student-m1-api
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$ docker run -d --name port-test -p 8080:5000 ghcr.io/usmanharoon1177/student-m1-api:1.1.0
1450b62262f2f1ab5516a0d30176249c085793bb1521c35228011155753
HTTP/1.1 200 OK
date: Wed, 09 Sep 2026 19:38:08 GMT
server: uvicorn
content-length: 187
content-type: application/json

{"status": "healthy", "application": "student-m1-api", "application_version": "1.1.0", "mode": "dev"}
usman@DESKTOP-K9H1088:~/OPS/AI/student-m1-api$
```

Case 1: Failed Unit Test (Pytest)

- **Symptom:** Pull request CI checks failed automatically, blocking merge eligibility with a red cross (⌘) on GitHub Actions.
- **Root Cause:** An intentional regression was introduced in tests/test_app.py, modifying the expected health endpoint status assertion from "healthy" to "wrong".
- **Evidence:**
 - GitHub Actions pull request view displaying a failed ci.yml run status.
 - Pytest execution traceback in CI logs showing AssertionError: assert 'healthy' == 'wrong'.
- **Correction:** Restored the assertion assert data["status"] == "healthy" in tests/test_app.py, committed with message fix: correct health endpoint test, and pushed to the branch to pass CI.

Case 2: Wrong Container Port Mapping

- **Symptom:** Running curl -i http://localhost:5000/health resulted in curl: (7) Failed to connect to localhost port 5000: Connection refused.
- **Root Cause:** The container was launched with an mismatched port flag (-p 6000:5000), routing container port 5000 exclusively to host port 6000. Host port 5000 remained unmapped with no process listening.
- **Evidence:**
 - Terminal command showing container executed with -p 6000:5000 followed by curl returning Connection refused on host port 5000.
 - Terminal output after executing docker run with -p

5000:5000, followed by curl returning HTTP/1.1 200 OK.

- **Correction:** Stopped the mismatched container and relaunched it using -p 5000:5000, properly aligning the published host interface with the containerized Uvicorn service.